



本源量子
Origin Quantum

Origin PilotOS 4.2 User Manual and API Reference Document

Origin Quantum Computing Technology (Hefei) Co., Ltd.

Version 3.4

April 29, 2026



Origin Pilot

本源司南

Contents

1	Introduction	1
1.1	Purpose of the Manual	1
1.2	System Overview	1
1.3	Terms and Abbreviations	2
2	Installation and Deployment	3
2.1	Hardware Requirements	3
2.2	Software Dependencies	4
2.3	Deployment Preparation	4
2.3.1	Extracting the Deployment Package	4
2.3.2	Installing Deployment Dependencies	4
2.4	Deployment Process	5
2.4.1	Database Initialization	5
2.4.2	Modifying Configuration Files	5
2.4.3	Loading Image File	6
2.5	Service Startup and Verification	6
2.5.1	Starting Services	6
2.5.2	Stopping Services	6
2.5.3	Verifying Access	7

3	System Architecture	8
3.1	Overall Architecture Diagram	8
3.2	Execution Workflow	9
3.3	Core Modules of the System Service Layer	11
3.3.1	QCloudServer Module	11
3.3.2	ComputerServer Module	12
3.3.3	OSServerManager Module	13
3.3.4	ScheduleServer Module	13
3.3.5	QCompileServer Module	14
3.3.6	ResourceManager Module	15
3.3.7	BackendServer Module	15
4	Visual Management Console	17
4.1	License Verification	17
4.2	User Account Management	18
4.2.1	User Registration	18
4.2.2	User Login	19
4.2.3	User Management	20
4.3	Dashboard Overview	21
4.3.1	Resource Usage	21
4.3.2	Module Status Cards:	22
4.3.3	Real-time Task Manager:	22
4.4	Resource Management Page Overview	22
4.4.1	Chip Management	23
4.4.2	Chip Details Module	25
4.4.3	Cluster Management Module	27
4.4.4	Upgrade PilotOS Functional Module	28
4.5	Task Management	30
4.5.1	Task Submission Page	30
4.5.2	My Tasks Page	33
4.5.3	Task Details Page	34
4.5.4	Real-Time Task Status Page	36
4.5.5	User Task Management Page	37
4.5.6	Task Report Page	38
5	API Reference	40
5.1	PilotOS Login	40
5.2	Submit Quantum Chip Computing Task	41

5.3	Query Task	44
5.4	Get Queue Information	45
5.5	Get Chip Information	46
5.6	Submit Cluster Simulation Compute Task	47
5.6.1	Configuration Info Description and Request Examples	48
5.7	Cluster Simulation Task Status Query	50
5.8	Get Task Queuing Info for Specified Backend	51
5.9	Get Chip Calibration Time	51
5.10	Pause Chip Scheduling Service	52
5.11	Get Chip Schedule State	52
5.12	Manual M3 Sampling	53
5.13	Session Mechanism Task Management	53
5.13.1	Request Session	54
5.13.2	Submit Task in Session	54
5.13.3	Close Session	55
5.14	Get Chip Best Qubit Blocks	55
5.15	Get Task Priority	56
6	Message Push	57
6.1	Chip Calibration Start Information	57
6.2	Chip Calibration End Information	57
6.3	Chip New Configuration Information	58
6.4	Chip Manual Maintenance Start Information	58
6.5	Chip Manual Maintenance End Information	59
7	Type Definitions and Error Codes	60
7.1	Task Types	60
7.2	Compute Backend Types	60
7.3	Task Status Types	61
7.4	Error Codes (ErrorCode)	61
8	Operations and Management	63
8.1	System Maintenance	63
8.2	Troubleshooting	63
8.3	Deployment FAQs	64
8.3.1	PilotOS Service Anomaly & PilotOS License Verification Service Temporarily Inaccessible	64
8.3.2	License Verification Server Connection Error, Please Check Network	68

1 Introduction

1.1 Purpose of the Manual

This user manual aims to provide comprehensive operational guidelines and technical details for the quantum operating system "Origin PilotOS". The primary audience includes the following groups:

- **Quantum Computing Research Teams:** Researchers focusing on quantum computing theory, algorithm optimization, and quantum hardware/software integration. They use PilotOS as an experimental platform to test and verify novel quantum algorithms or architectures.
- **Quantum Information Scientists:** Researchers in fields such as quantum cryptography, quantum communication, and quantum networks, utilizing PilotOS to manage and orchestrate complex experimental tasks.
- **Quantum Application Developers:** Individuals and teams developing quantum programs or tools, utilizing PilotOS for efficient programming, debugging, and algorithm testing.
- **Tech Companies and Startups:** Enterprises focusing on quantum hardware development or quantum software platforms, using PilotOS to manage hardware resources, optimize performance, or serve as the infrastructure for providing quantum computing services.
- **National Laboratories and Policymakers:** Government-funded research labs utilizing PilotOS to advance quantum technology research and support national innovation strategies.

This manual aims to help these users effectively understand, operate, and manage the PilotOS system.

1.2 System Overview

Quantum computing is a novel computing paradigm combining quantum mechanics and computer science. It uses qubits as the fundamental unit of computation, leveraging quantum phenomena such as superposition, entanglement, and measurement to achieve exponential acceleration over classical computing on specific problems. Quantum computing has significant advantages in fields like breaking asymmetric encryption, quantum chemistry, financial modeling, and machine learning.

In recent years, the scale and quality of quantum chips have developed rapidly. The latest superconducting quantum chips have reached 443 qubits (e.g., IBM's "ibm-seattle" chip), and it is expected that superconducting chips will exceed 1,000 qubits in the coming

years. Meanwhile, quantum chip technologies based on different physical platforms are developing in parallel, each with unique advantages in computational efficiency, fidelity, and stability.

Managing quantum computing resources at a large scale of qubits, scheduling tasks efficiently, and achieving heterogeneous multi-backend quantum computing and quantum-classical hybrid computing are critical challenges that must be addressed in the development of practical quantum computing systems. PilotOS is China's first quantum computing operating system. Its core goal is to enable highly efficient quantum-classical hybrid computing. The system consists of key components including quantum circuit compilation and optimization, quantum resource management, quantum task scheduling, system monitoring, and backend quantum computing services. It supports integrated computing across heterogeneous quantum backends and provides users with full-stack quantum computing services through the open-source quantum programming framework **PyQPanda**.

1.3 Terms and Abbreviations

The following are key technical terms and abbreviations used in this manual:

- **QPU (Quantum Processing Unit):** The quantum counterpart to a classical CPU, responsible for executing quantum operations on quantum hardware.
- **NISQ (Noisy Intermediate-Scale Quantum):** Refers to the current era of quantum computing, characterized by devices with tens to hundreds of qubits that are susceptible to noise and errors, limiting their computational reliability and scalability.
- **Qubit (Quantum Bit):** The fundamental unit of quantum information, capable of existing in a superposition of the $|0\rangle$ and $|1\rangle$ states[cite: 5].
- **Quantum Superposition:** The principle that allows a quantum system to exist in multiple states simultaneously until it is measured.
- **Quantum Entanglement:** A phenomenon where two or more qubits become correlated, such that even when separated by large distances, the state of one cannot be described independently of the others.
- **PyQPanda:** An open-source quantum programming framework that allows users to write, simulate, and execute quantum programs on various quantum backends.

2 Installation and Deployment

This chapter details the automated deployment process of the PilotOS system. Automated deployment is primarily based on the automation script "AutoDeploy.py", which encapsulates image management and database initialization steps. Users only need to execute this script to complete the entire installation operation.

Note: If you have already installed PilotOS and need to upgrade to the current version, please refer to the "PilotOS Upgrade Instructions".

2.1 Hardware Requirements

Hardware mainly involves the following categories of components:

- **CPU**
 - Recommended to use AMD Zen 5 series, Intel Arrow Lake / Diamond Rapids series, or newer models;
 - To fully unleash the performance potential of PilotOS, it is recommended to use at least a 4-core processor; if installing a virtual machine, it needs to be allocated to the virtual machine;
- **Memory (DRAM)**
 - Recommended capacity of no less than 64 GB;
 - Recommended to use DDR5, frequency ≥ 5600 MHz;
 - Powerful CPU performance requires sufficient memory bandwidth and capacity.
- **Storage**
 - Hundreds of GB of disk space are required during installation and operation;
 - It is recommended to reserve at least 1 TB of storage capacity for PilotOS[cite: 6].
- **Network Interface Card (NIC)**
 - Required bandwidth depends on the workload scale;
 - Due to the exponential growth of the quantum state space, a NIC bandwidth of at least 10 Gbps is recommended;
 - To avoid the network becoming a bottleneck for overall computational efficiency.

2.2 Software Dependencies

Before beginning deployment, please ensure the host machine meets the following software environment requirements:

- **Operating System:** Ubuntu Server 22.04 LTS recommended
- **Python Environment:** Python $\geq$ 3.8.0
- **Container Engine:** Docker $\geq$ 20.10.0 (requires the docker-compose component installed)
- **MySQL Database Service:** Version 8.0 or above (must support remote access)
- **MongoDB Database Server:** Version 6.0 or above (must support remote access)

2.3 Deployment Preparation

The file structure after extracting the PilotOS deployment package is as follows:

Listing 1: Deployment Package File Structure

```
1 ./package
2 +-- AutoDeploy.py # Automated deployment script
3 +-- docker-compose.yml
4 +-- mysqlInit # Database initialization scripts directory [cite: 14]
5 +-- pilotos:x.x.iso # PilotOS image file for version x.x
6 +-- PilotOS-Config # Configuration files directory
7 +-- requirements.txt # Python dependency list [cite: 19]
8 +-- Upgrade.py
9 +-- Util.py
```

2.3.1 Extracting the Deployment Package

Execute the following command to extract the deployment package (please modify according to the actual file name):

```
1 tar -zxf pilotos-x.x.tar.gz
```

2.3.2 Installing Deployment Dependencies

Use pip to install the Python dependency libraries required for the deployment script:

```
1 sudo pip3 install -r requirements.txt
```

Note: You must use `sudo pip` for installation to ensure dependencies are installed in the system environment. When executing the automated script subsequently, you must

also use `sudo python`, otherwise it will report an error stating the dependency package cannot be found.

2.4 Deployment Process

2.4.1 Database Initialization

Before executing initialization, please first modify the database configuration in `config.yaml` under the `mysqlInit` folder, changing it to the database configuration of the environment you intend to deploy.

```
1 vim mysqlInit/config.yaml
```

After modification, execute the following in the extracted root directory:

```
1 sudo python3 AutoDeploy.py initDB [cite: 24]
```

2.4.2 Modifying Configuration Files

According to the actual deployment environment, modify the MySQL and MongoDB configurations in the following configuration file. The MySQL address should be the database initialized in step 2.4.1:

```
1 sudo vim ./PilotOS-Config/config.json
```

Note: MySQL and MongoDB must support remote access. The database address cannot be **127.0.0.1**; it must be a specific IP address.

If you need to support account registration to allow multiple users, you must configure an email server. Configure the email settings in `config.json`, where `recv_accounts` represents the email addresses receiving PilotOS exception messages, as follows (using QQ mail as an example):

```
1 "email": {
2     "username": "xx@qq.xx",
3     "pwd": "xx",
4     "server_url": "smtp.qq.com",
5     "port": "465",
6     "ssl_trust": "smtp.qq.com",
7     "recv_accounts": [
8     ]
9 }
```

2.4.3 Loading Image File

Execute the following command to load the PilotOS Docker image:

```
1 sudo python3 AutoDeploy.py loadImage
```

Note: Before executing this command, you should ensure that the Docker container engine (version 20.10.0 or above) and the docker-compose component have been correctly installed in the system.

2.5 Service Startup and Verification

2.5.1 Starting Services

Execute the following command to start the PilotOS image service and background upgrade service:

```
1 sudo python3 AutoDeploy.py start
```

Startup Notes:

- The script will first check whether the configured MySQL and MongoDB can be connected correctly. If connection fails, it will prompt in the command line and will not continue the startup process.
- Before each startup, the script automatically checks whether the service is already running. If it exists, it will not restart it[cite: 25].

2.5.2 Stopping Services

Stopping the PilotOS service requires stopping the image service and the upgrade service. **Note: Stopping the image container will delete the modifications you have made in the image!**

Stop the image service: Execute the following command under the pilotos folder:

```
1 sudo docker compose down
```

Stop the upgrade service: Because the upgrade service executes in the background as a daemon process, you need to get the process ID and then kill the service.

```
1 ps aux | grep AutoDeploy.py
2 sudo kill <Process ID>
```

Shutdown Notes:

- **Terminating Upgrade Service Instructions:** If it is necessary to shut down the upgrade service under special circumstances, you **must** use `sudo kill` to kill the process. It is **strictly prohibited** to use `kill -9`.

- Using `kill -9` will cause the `/var/run/autodeploy_upgrade.pid` file to not be deleted, thereby causing the next startup to fail. The upgrade service can only start normally when this PID file does not exist.

2.5.3 Verifying Access

After the service has started successfully, you can test it by accessing the management console through a browser:

- **Access Address:** `http://<Server IP>:10020`

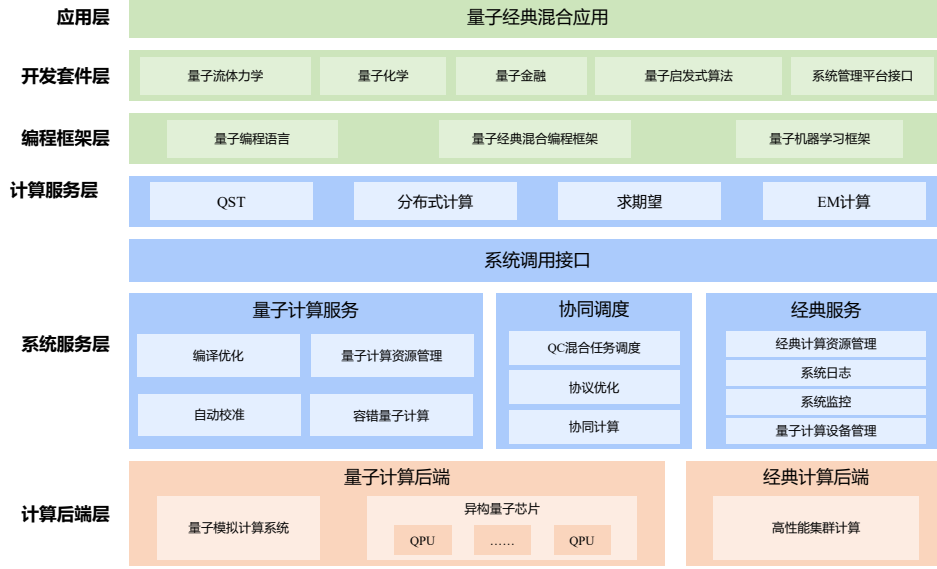
Administrator account: Username: admin Password: admin123

If you encounter problems, refer to [Chapter 8.3](#).

3 System Architecture

3.1 Overall Architecture Diagram

The overall architecture of Origin PilotOS is shown in the figure below.



Application Layer: Primarily refers to the user programming space, where users can develop various applications on top of the Origin PilotOS system.

Application Development Kit: Origin PilotOS provides upper-layer users with various development kits, which have been integrated into the Pilot system. After deploying the PilotOS system, users can directly use these kits for secondary function development or rapid verification of quantum algorithms. The tool suites currently included are: ChemiQ (Quantum Computational Chemistry Package), QFinancial (Quantum Finance Development Kit), QCFD (Quantum Computational Fluid Dynamics Development Kit), Quantum Heuristic Algorithm Toolkit, and System Manager Interface.

Built-in Programming Frameworks: The Origin PilotOS system provides three integrated programming frameworks:

a. Quantum Programming Framework. [cite: 26] A framework dedicated to quantum program development, providing: 1) High-level quantum programming languages; 2) Simulation interfaces for executing and verifying quantum algorithms; High-level quantum circuit tools, including circuit printing, equivalence checking, circuit inversion, and conversion to other quantum programming languages (e.g., Qiskit, Cirq).

b. Quantum-Classical Hybrid Compilation Framework. Optimizes hybrid quantum-classical programs by: 1) Analyzing and scheduling the execution order between

quantum and classical tasks; 2) Applying compiler-level optimizations to improve overall algorithm efficiency and resource utilization.

c. Quantum Machine Learning (QML) Framework. Supports seamless development of quantum and hybrid machine learning models, featuring: 1) Python-based user-friendly API, supporting automatic differentiation and dynamic computational graphs; 2) Unified execution across multiple hardware backends (quantum processors, CPUs, GPUs); 3) Built-in high-performance numerical libraries to accelerate training and inference; 4) Compatibility with mainstream machine learning frameworks (e.g., PyTorch, TensorFlow) for easy integration into existing ML workflows.

System Interfaces: The unified gateway for upper-layer applications to access the Origin PilotOS system. It enforces isolation between the system layer and the application layer, ensuring that user-space applications can only access system resources in a limited, controlled manner, thereby maintaining system stability and security.

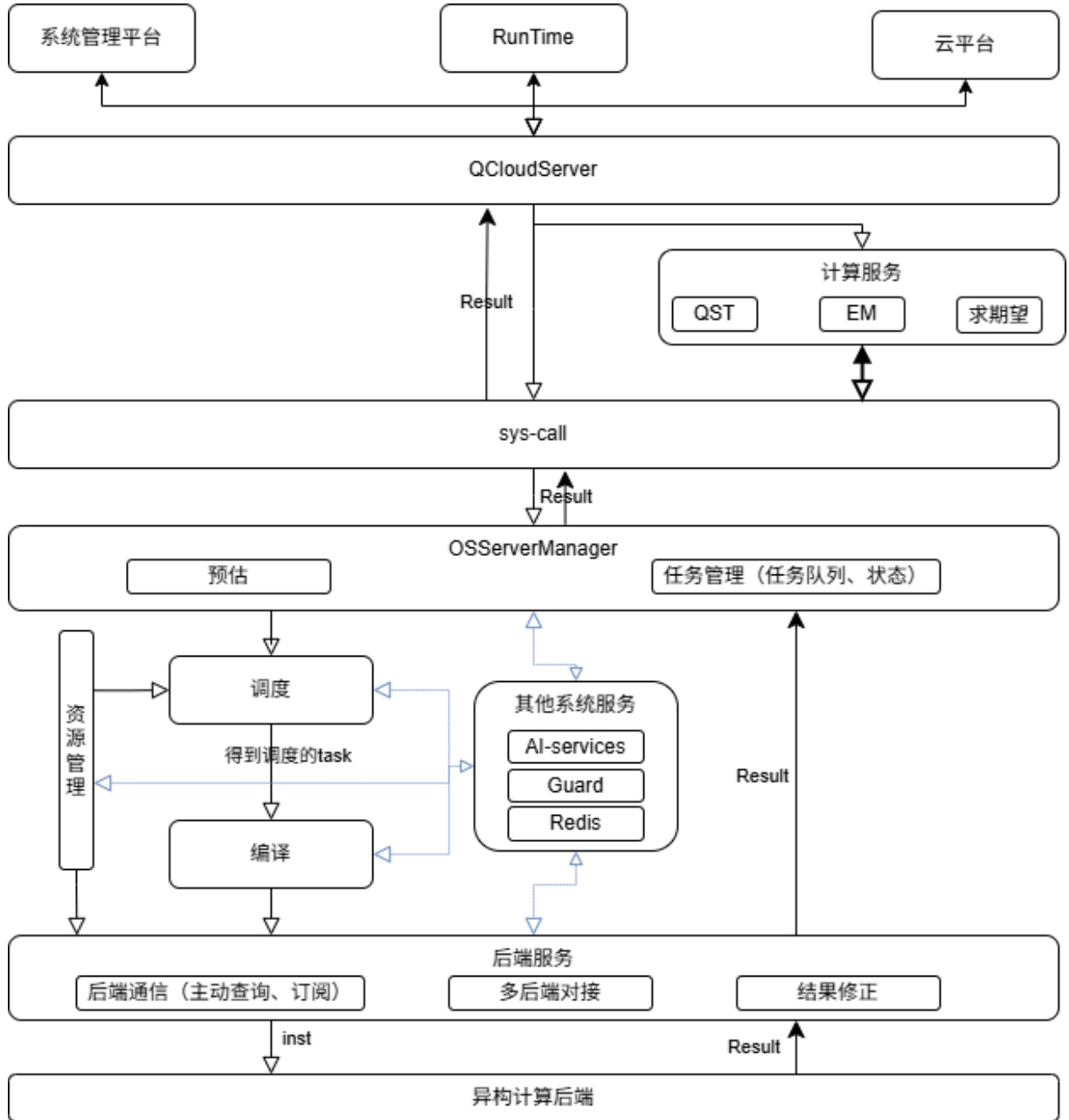
Compute Service Layer: Handles specific business scenarios with dedicated processing logic, decoupling application-level compute tasks from core system computation to ensure the stability of overall calculations. Currently integrated services include: 1) Quantum State Tomography (QST), 2) Expectation Value Estimation, 3) Error Mitigation (EM) calculations, and 4) Distributed Quantum Circuit Computing.

System Service Layer: The Origin PilotOS quantum operating system is built on a set of core, interoperable modules: System Management Module, Scheduling Service Module, Compilation Service Module, Resource Management Module, and Backend Service Module. These modules communicate seamlessly via secure internal system channels to achieve coordinated workflow execution—from task submission and compilation to resource allocation and quantum computation on physical hardware. This architectural design focuses on separation of concerns and loose coupling, ensuring high reliability, scalability, and compatibility across heterogeneous quantum backends.

Compute Backends: Origin PilotOS natively supports various quantum computing backends, including superconducting quantum processors, quantum simulators, and classical computing backends.

3.2 Execution Workflow

The following figure illustrates the end-to-end workflow for quantum computing tasks in the Pilot system.



Task Submission: Users can submit quantum computing tasks via the following methods:

- **QPandaLite:** A Python SDK built on top of the Pilot Web management interface (via Nginx service);
- **PilotOSMachine:** Available in Python and C++ versions;
- **Origin Cloud Platform:** Public HTTP APIs for external access.

All tasks are received and persistently managed by the QCloudServer layer.

Task Preprocessing and Routing: For dedicated tasks (such as Quantum State Tomography (QST), expectation estimation, EM calculations), QCloudServer forwards them to dedicated compute services for preprocessing before invoking the Pilot kernel via

system APIs.

Task Orchestration in Pilot:

- The OSServerManager module acts as a central coordinator, managing internal modules and maintaining task queues.
- The state of each task is tracked via a shared state mapping table. As tasks flow between modules, their status updates are aggregated in OSServerManager.
- Upon task completion (success, failure, or cancellation), all state data is transmitted to QCloudServer for persistent storage and removed from OSServerManager—ensuring OSServerManager only retains the states of active tasks.
- OSServerManager also provides resource estimation, predicting the required number of qubits, circuit depth, and runtime to support scheduling decisions.

Scheduling: Tasks that have undergone resource estimation enter the scheduling queue. The scheduler allocates resources based on the following factors:

- Task resource/runtime requirements,
- Current backend availability.

Supported strategies include:

- First-Come, First-Served (FCFS),
- Highest Response Ratio Next (HRRN),

These can be configured via Pilot’s configuration file.

Compilation and Execution: Scheduled tasks are compiled into backend-specific instructions through optimization passes. The backend service then:

- Distributes tasks to the target quantum hardware (supporting multiple architectures),
- Retrieves results and backend status,
- Applies result correction algorithms to mitigate hardware noise and improve accuracy.

This streamlined architecture ensures stability, scalability, and highly efficient resource utilization across diverse quantum computing workloads.

3.3 Core Modules of the System Service Layer

3.3.1 QCloudServer Module

The Pilot QCloudServer module serves as the only official gateway for users to access the Pilot quantum operating system[cite: 27]. It exposes a secure HTTPS-based API interface, ensuring all communication is encrypted and reliable.

- **Unified Access Gateway.** All user requests—including quantum task submissions, status queries, and result retrievals—must be routed through the Pilot cloud service. No other direct access paths to the system are exposed, to ensure security and centralized control.
- **Request Validation and Security Enforcement.** Upon receiving a request, the service verifies user identity, API permissions, and request parameters. Malformed, unauthorized, or potentially malicious requests are rejected before reaching core system components.
- **Persistent Task Management.** Validated requests are persistently stored in the cloud service’s database, including task metadata such as User ID, submission timestamp, task type, and configuration parameters. This enables full-lifecycle tracking, auditing, and recovery.
- **Secure Internal Routing.** Tasks and queries are forwarded through secure internal system channels to downstream Pilot modules (such as QCloudServer, OSServerManager) for compilation, scheduling, execution, or state lookup.
- **Result Delivery and State Synchronization.** Once completed, compute results or query responses are relayed back to the cloud service through the same internal channels before being delivered to the user via the HTTPS API—completing the end-to-end workflow.

3.3.2 ComputerServer Module

The ComputerServer module is situated between the PilotOS cloud service layer and the system kernel layer, acting as a middleware module. Its primary purpose is to decouple common but less frequently used computational functions from the core execution flow. This design not only reduces the load on the system kernel but also enhances overall scalability and functional agility. Currently, the computing service provides the following key capabilities:

- **Expectation Value Evaluation** for quantum circuits;
- **Full-Circuit Fidelity Estimation**;

This service utilizes a modular architecture composed of four core components:

- **Task Preprocessing:** Parses incoming task requests, validates input parameters, and generates standardized internal task representations;
- **Subtask Expansion:** Automatically decomposes high-level tasks into multiple subtasks that can be executed in parallel or sequentially, according to semantic requirements;

- **Subtask Execution:** Distributes subtasks to underlying quantum or classical compute resources for actual computation;
- **Result Aggregation:** Combines subtask outputs into a final user-facing result through normalization, weighting, or statistical fusion.

This framework is designed for easy expansion, seamlessly integrating new domain-specific computational capabilities while maintaining loose coupling with the main Pilot system.

3.3.3 OServerManager Module

The OServerManager module acts as the central coordination component within the Pilot system. It communicates with upper-layer applications through standardized system call interfaces (SysCall) and is responsible for two critical functions: task resource estimation and end-to-end task queue management.

- **Task Resource Estimation:** Before a quantum task is queued for execution, OServerManager performs static analysis on the submitted quantum circuit to estimate its resource requirements. This includes: the required number of qubits, circuit depth, and quantum gate count. Based on this analysis, the system determines whether sufficient resources are currently available to accept the task.
- **Task Queue Management:** Tasks that pass resource estimation are placed into a unified scheduling queue. OServerManager continuously tracks the progress of each task throughout the execution flow and updates its status in real-time. These status updates are immediately pushed to your application, allowing fine-grained visibility into every stage of task execution.
- **Persistent Storage of Task Data:** Upon task completion, Pilot automatically uses a tiered storage strategy to persist its data: structured metadata (e.g., Task ID, Backend ID, start/end timestamps) is stored in MySQL for efficient querying and monitoring. Large unstructured data (e.g., complete quantum circuits, raw measurement results) are saved in MongoDB to support flexible access and scaling. This design ensures data reliability and auditability while keeping the core system lightweight and responsive.

3.3.4 ScheduleServer Module

The ScheduleServer module receives tasks that have passed resource estimation—along with their associated resource requirements—and distributes eligible tasks to the compilation module for processing. By default, the system uses the Highest Response Ratio

Next (HRRN) scheduling policy, which prioritizes tasks based on wait time and estimated execution duration, thereby balancing fairness and throughput.

Scheduling Workflow:

- **Task Enqueueing:** Once a task passes resource estimation in `OSServerManager`, it is assigned an initial priority. This value is determined by `OSServerManager` based on configurable policies (e.g., task type, user tier, or business context)—it is not fixed at zero. The task is then pushed into the scheduler’s priority queue, implemented as a max-heap, ensuring that the task with the current highest response ratio is always at the top.
- **Dynamic Priority Updates:** The effective scheduling priority of a task is controlled by its response ratio, which increases as the task’s wait time in the queue increases, allowing it to gain higher priority over time. This prevents tasks with long wait times from being starved.
- **Scheduling Decision Logic:** During each scheduling cycle, the system executes the following steps:
 - 1) Checks if the resource requirements of the task at the top of the heap can be met by the currently available resources.
 - 2) If resources are sufficient: The task is popped from the queue and forwarded to the compilation module.
 - 3) If resources are insufficient: No tasks are distributed; instead, the response ratios (priorities) of all queued tasks are updated.
 - 4) Regardless of the outcome, the priority queue is refreshed at the end of the cycle to prepare for the next scheduling event.

3.3.5 QCompileServer Module

Origin PilotOS provides a comprehensive, flexible, and high-performance quantum circuit compilation and optimization framework. This framework abstracts the complexities of the underlying quantum hardware, allowing quantum software developers to focus on algorithm design and application innovation—without needing to understand the specific details of different quantum processor architectures.

Key Advantages:

- **Hardware-Agnostic Development:** Developers simply submit standard quantum circuits (e.g., QASM or internal intermediate representations)[cite: 28]. The compiler automatically adapts the circuits to the target backend, enabling "write once, run anywhere" capability across supported quantum hardware platforms.
- **Heterogeneous Multi-Backend Support:** The current version supports two

mainstream quantum computing modalities: superconducting and ion trap processors. Depending on the user-specified target backend, the system dynamically selects the appropriate compilation pipeline to generate a fully compatible native instruction set.

- **Parallel Compilation Acceleration:** Built upon a parallel compilation framework, the service can process multiple tasks concurrently or decompose large circuits into fragments for parallel optimization—significantly improving overall compilation throughput, especially for batch jobs or highly complex circuits.

Compilation and Optimization Capabilities: The Pilot compiler integrates a suite of advanced optimization algorithms, each encapsulated via standardized, modular interfaces to promote reuse, composition, and future expansion. Core optimization techniques include:

- **Multi-Controlled Gate Decomposition**
- **Gate Set Conversion**
- **Quantum Circuit Optimization**
- **Qubit Mapping and Topology Routing**

3.3.6 ResourceManager Module

The Resource Management module is responsible for receiving resource update notifications from quantum computing backends or resource configuration messages from upper-layer system components. It maintains an up-to-date inventory of available resources and synchronizes this information with other services in the system.

Core Responsibilities:

- **Handling Resource Updates and Configuration Requests:** Receives messages regarding resource changes (e.g., backend availability, qubit status) or explicit resource configuration commands, and updates the internal resource registry accordingly.
- **Serving Resource Information Queries:** Responds to requests from other system components, providing current and accurate resource data on demand.

3.3.7 BackendServer Module

The BackendServer module is responsible for managing multiple heterogeneous quantum computing backend units (e.g., superconducting, ion trap) and handling the distribution of computational tasks and receipt of execution results. Serving as the central communication hub between the Pilot system and the underlying quantum control

services, this module utilizes a hybrid messaging model to balance reliability, real-time responsiveness, and scalability.

Communication Architecture: To meet diverse operational requirements, the backend service module integrates two ZeroMQ communication patterns when interacting with quantum control services:

a. Router-Dealer Pattern (Request-Reply Pattern) [cite: 29]

- **Characteristics:** Connection-oriented, bidirectional, synchronous request-response.
- **Primary Use Cases:** Distributing quantum computing tasks to specific backends; receiving task execution results; querying real-time task status; retrieving Randomized Benchmarking (RB) experimental data; exchanging heartbeat signals for link health monitoring and fault detection.
- **Advantages:** Ensures orderly, reliable message delivery—ideal for critical control commands and result retrieval.

b. Pub-Sub Pattern (Publish-Subscribe Pattern) [cite: 30]

- **Characteristics:** Connectionless, unidirectional, asynchronous, event-driven.
- **Role:** The Pilot backend service acts as a subscriber, continuously listening for broadcast messages from control services.
- **Push Events:** Typical push events include: proactive task status updates (e.g., abnormal termination, resource preemption); dynamic changes in chip parameters (e.g., T1/T2 coherence times, gate fidelity); notifications upon completion of automatic calibration routines; real-time updates on chip resource availability (e.g., qubit failures, cooling recovery).
- **Advantages:** Eliminates polling overhead and enables low-latency event propagation, enhancing system responsiveness.

4 Visual Management Console

This chapter provides an overview of the PilotOS Visual Management Console, including its interface structure and core functional modules. The visual management console provides an integrated visual platform covering user account management, system monitoring, resource scheduling, and full task lifecycle operations, enabling users and administrators to efficiently operate quantum computing and cluster resources.

4.1 License Verification

Before using the PilotOS system for the first time, users must complete license verification. The license authorizes overall access to the system and is a prerequisite for entering the platform.

When the system has not recorded a license, or the current license has expired, it will automatically route to the license input page, as shown in Figure 1. Users must enter a valid license on this page before continuing to use the system.

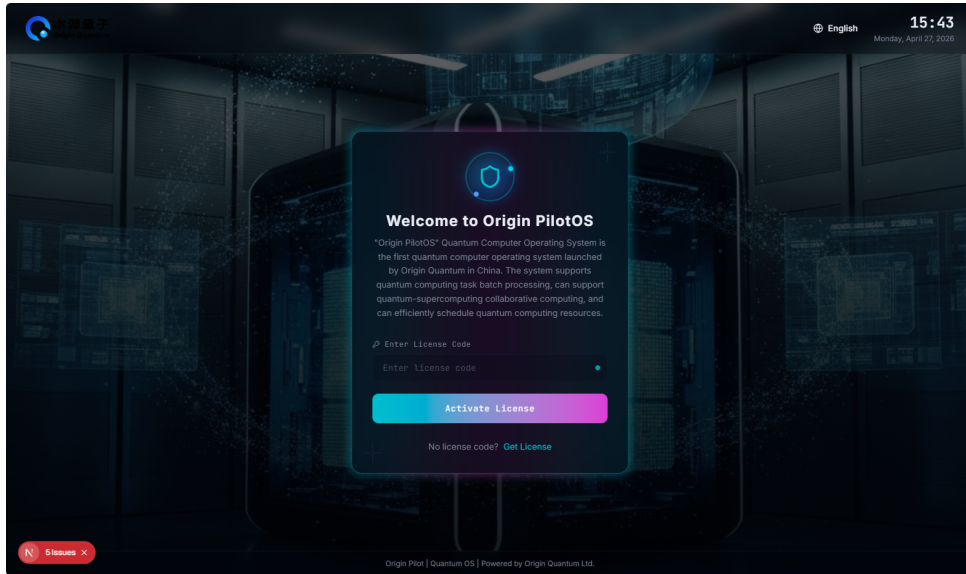


Figure 1: License Input Page

Usage Process When accessing the system for the first time or when the license has expired, users need to complete verification following these steps:

1. Enter a valid license code on the license page;
2. Click the confirm button to submit;
3. Upon successful verification, proceed to the system login page;
4. Log in to use system functions normally.

If the entered license is invalid or expired, the system will display a prompt, and the user must re-enter a valid license.

License Status Description From the user’s perspective, licenses primarily exist in the following use scenarios:

- **License Not Entered:** A license must be entered during the initial system use; otherwise, access to the system is denied.
- **License Valid:** [cite: 32] System functions are fully open, and users can use the platform normally.
- **License Expiring Soon:** The system is still usable, but a prompt will show the remaining valid time on the interface. Early renewal is recommended.
- **License Expired:** The system will suspend all functions, only allowing the entry of a new license. Functions resume after a valid update.

The top of the system’s main interface will display the remaining valid days for the license to help users stay aware of the current authorization status.

Notes

- It is recommended to update the license before expiration to avoid service interruptions;
- Once the license expires, a valid license must be re-entered to restore system functionality;
- If you are unable to complete license verification, please contact the system administrator or operations personnel.

4.2 User Account Management

4.2.1 User Registration

Description: This feature allows users to register a new account for the PilotOS visual management console.

Registration Process: To complete registration, users must accurately provide the following information. Please note that duplicate account registration is not allowed.

Form Fields:

- **Full Name.**
- **Organization.**
- **Account** – Provide a unique account. This account will be used as your login identifier.

- **Password** – Create a secure password that meets system requirements.

The image shows a registration form for the 'Origin Pilot Quantum Computing Platform'. The form is centered on a dark background with a faint image of server racks. The form has a title 'Create Quantum Account' and a subtitle 'Join the Origin Pilot Quantum Computing Platform'. It contains five input fields: 'Real Name', 'Organization', 'Account', 'Register Password', and 'Confirm Password'. Each field has a placeholder text and a small icon. Below the fields is a large 'Register' button with a gradient from blue to purple. At the bottom of the form, there is a link 'Already have an account? Login now'. The top right corner of the interface shows 'English' and '15:45 Monday, April 22, 2026'. The bottom left corner has a red notification bubble that says '1 issue'. The bottom center has the text 'Origin Pilot | Quantum OS | Powered by Origin Quantum Ltd.'.

Figure 2: Registration Form Interface

After completing all required fields, click the **Register** button to submit the registration request. The system will validate the inputs and create your account if all conditions are met.

Note: The email address must remain unique in the system. Attempting to register with an already registered email will result in an error prompt.

4.2.2 User Login

Function Description: This function allows users to log in to the PilotOS Visual Management Console using their registered account credentials.

Login Process: To access the system, users must input valid login information and submit the form. Upon successful verification, the system will redirect the user to the main dashboard. Incorrect credentials will display an error message prompt.

Required Form Fields:

- **Account** – Enter the Account Name associated with the registered account.
- **Password** – Enter your account password.

Administrator Account: username: admin password: admin123

Upon successful login, the system will display the dashboard interface.

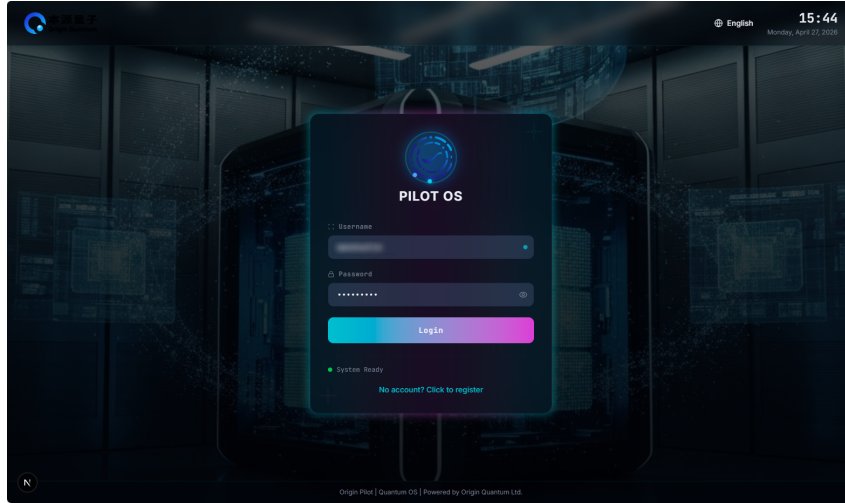


Figure 3: Login Form Interface

4.2.3 User Management

Function Description: This function allows administrator accounts to manage user information and control user access rights within the PilotOS Visual Management Console. Administrators can modify account status, adjust permissions, and perform necessary account maintenance operations.

Operation Flow: Once logged in, administrators can access the **User Management** page from the side navigation menu. The user management page allows administrators to filter and display user information based on specific criteria. Filter options include searching by **Email**, **Python API Access Permission**, and **User Role** (Regular User or Admin).

Available Actions:

- **Adjust Permissions** – Use the sliders under the **VIP**, **Set as Admin**, and **Python API Access Permission** columns to enable or disable respective user privileges. Adjusting these sliders controls whether a user has VIP privileges, admin privileges, or Python API access permissions.
- **Reset Password** – Click the **Reset Password** button to reset a user's password to the system default 123456.
- **Delete Account** – Click the **Delete** button to permanently remove the selected user account from the system.

Administrators can dynamically manage user roles and permissions by adjusting the sliders in the table. For example, enabling **Set as Admin** will grant administrator privileges, while toggling **VIP** or **Python API Access Permission** will control access to corresponding features.

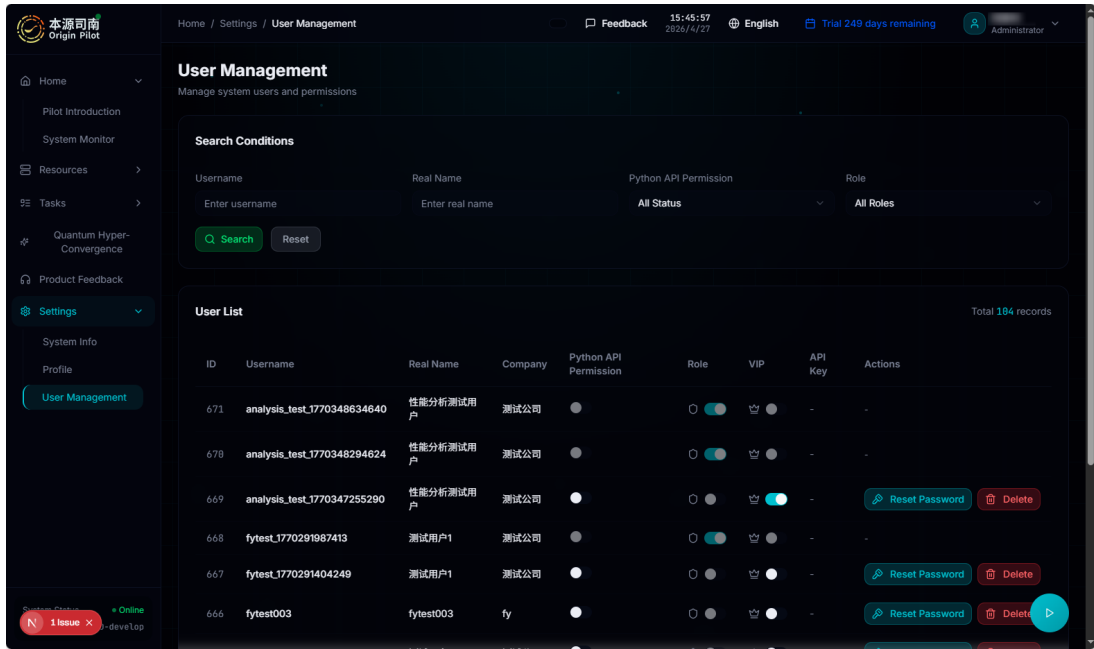


Figure 4: User Management Interface

Note: Only Administrator accounts can access the **User Management** page. Regular users do not have the right to view or modify other users' information.

4.3 Dashboard Overview

Page Overview: The dashboard page serves as a comprehensive overview of the PilotOS visual management console. It displays current resource usage, system health monitoring, and the real-time task manager (see Figure 5), allowing users to quickly understand the overall system status and real-time conditions.

4.3.1 Resource Usage

This section displays the following metrics in real time:

- PilotOS service CPU usage
- PilotOS service memory usage
- PilotOS host CPU usage
- PilotOS host memory usage

All indicators are refreshed periodically to reflect the real-time operating status of the system.

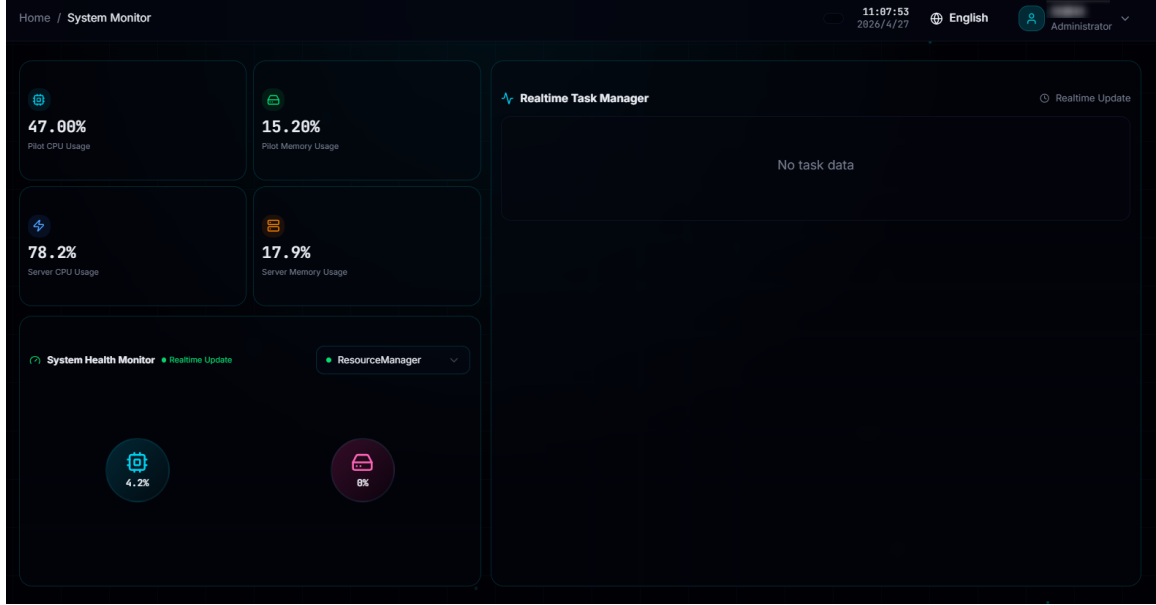


Figure 5: System Dashboard Main Interface.

4.3.2 Module Status Cards:

This section shows the real-time status of each Pilot system module. Users can switch between modules using the dropdown menu. The displayed information includes:

- Module name
- Module CPU usage
- Module memory usage
- Service status (**Normal** / **Abnormal**)

The service status is dynamically determined based on the module CPU usage and is visually represented using color labels.

4.3.3 Real-time Task Manager:

The table on the right displays a summary of currently running tasks, including:

- Task ID
- Submission time
- Task status

4.4 Resource Management Page Overview

Page Overview: The Resource Management page is the core interface of the visual management console for unified monitoring and configuration of quantum chips and computing clusters. This page provides a centralized view of the operational status, basic configurations, and scheduling information of all computing resources (see Figure 6). The

Resource Management main page consists of two sections — **Chip Management** and **Cluster Management**.

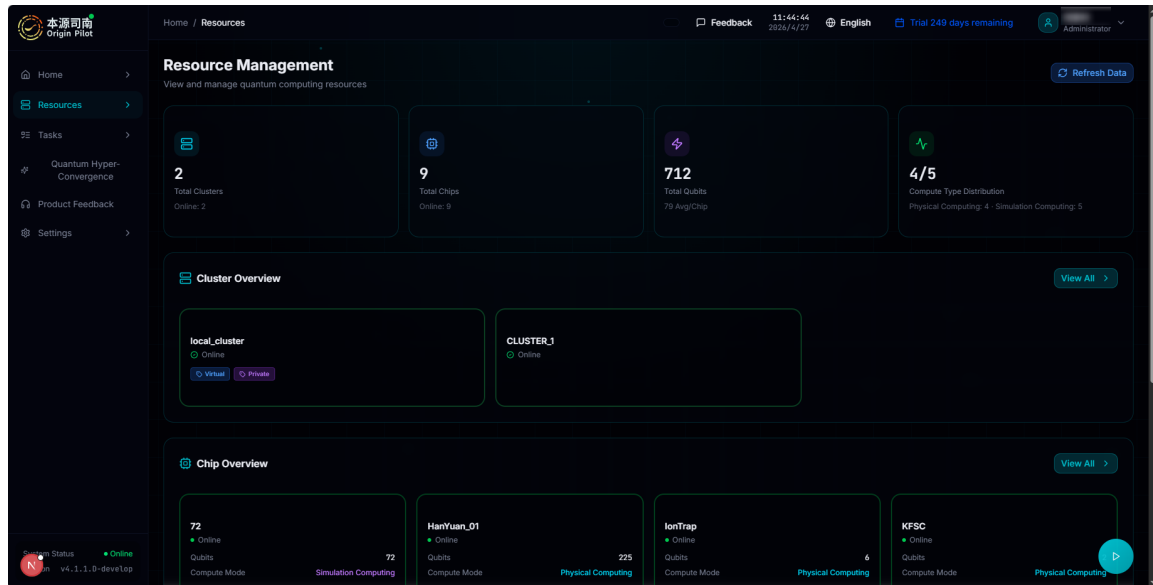


Figure 6: Resource Management Page Main Interface

4.4.1 Chip Management

Function Description: The Chip Management module provides a comprehensive overview of all quantum chips currently registered in the PilotOS system. Each chip card shows real-time metrics and allows administrators to directly control scheduling operations from the interface. In addition to automatically registered chip resources, this module also supports users to manually add custom chip resources. Users can input basic chip information via the "Add Resource" entry, achieving rapid access for new computing resources; It also supports editing and deleting existing chip resources. As shown in Figure 7, each chip is displayed as a separate card showing the chip name, configurations, and current running status and other key information. Edit and delete actions are provided below each card.

After clicking the "Add Resource" button, a chip resource creation form will pop up, as shown in Figure 8. Users fill in chip details (chip name, type, etc.) according to page prompts, and submit to complete resource registration.

Operations Guide:

- Click a chip card to open the detail drawer to view extended configurations and performance data.
- Click the **Pause Run** or **Resume Run** button on the chip card to control task scheduling.

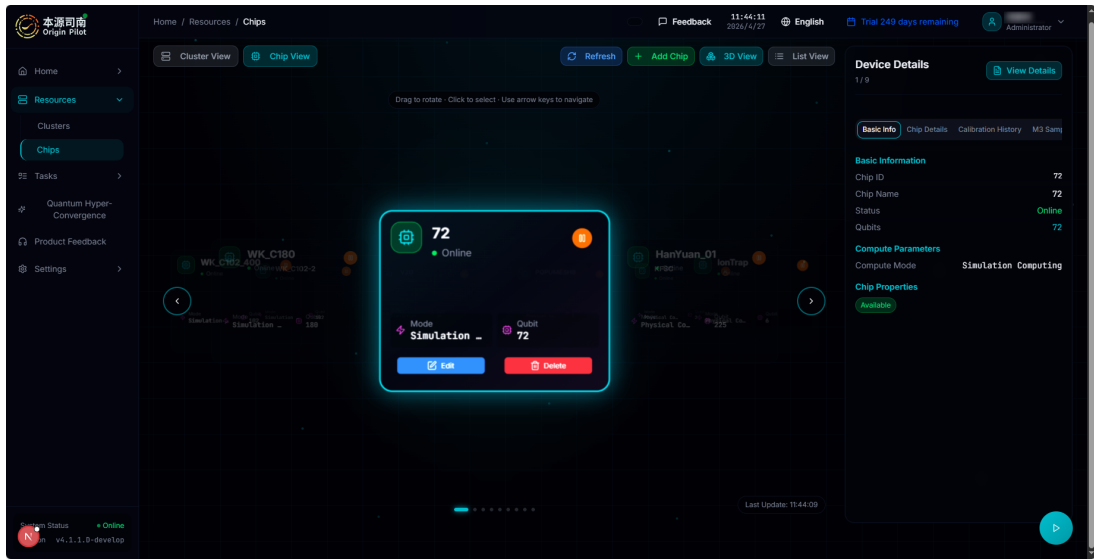


Figure 7: Chip cards display in the Chip Management Tab

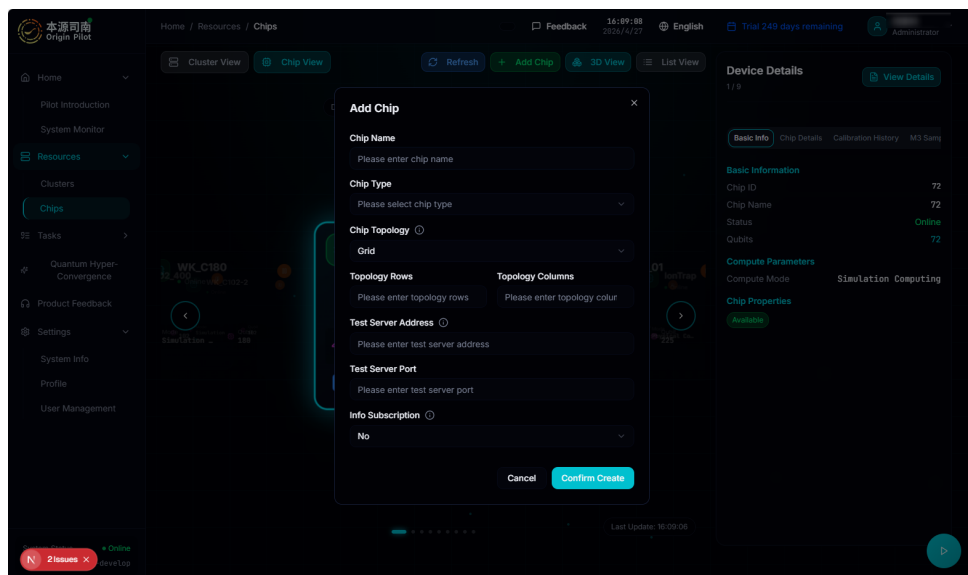


Figure 8: Add Chip Resource Form View

- Click the **Add Resource** button at the top of the page, fill in the chip info according to form prompts and submit to complete new resource access.
- Use the **Edit** button below the chip card to update existing resource configurations.
- Use the **Delete** button below the chip card to remove retired chip resources.

4.4.2 Chip Details Module

Function Description: The Chip Details module provides a comprehensive interface for viewing, analyzing, and managing quantum chip configurations and operational data. When a user clicks on a chip card, a details drawer slides out from the right, containing two functional tabs: **Information** and **Configuration**, as shown in Figure 9.



Figure 9: Chip Details Drawer showing Information and Configuration tabs.

Information Tab This tab acts as the visual main interface for quantum chip topology, qubit parameters, calibration history, and system configurations.

- **Top Area:** [cite: 35] Displays chip name, qubit count, and topology type, and provides:
 - **Push Config to Cloud** – Manually sync configuration data to the cloud.
 - **Download Config** – Export current chip settings.
- **Topology View:** Interactive topology map showing qubits and connection relationships. Supports hover or click to view data, clicking a qubit automatically locates the corresponding entry in the parameter table.

- **Qubit Parameters Table:** Lists Frequency, T_1 , T_2^* , readout fidelity, and single-qubit gate fidelity, with high-frequency qubits automatically marked.
- **Chip Properties:** Displays logic gate timing, noise models, and system metadata., as shown in Figure 10:



Figure 10: Bottom tabs showing operation history and chip attributes.

Configuration Tab Admin-only access, used for advanced configuration and scheduling management (see Figure 11). **Configurable Options:**

- Chip event notification settings (timed email alerts)
- Exclusive machine time management

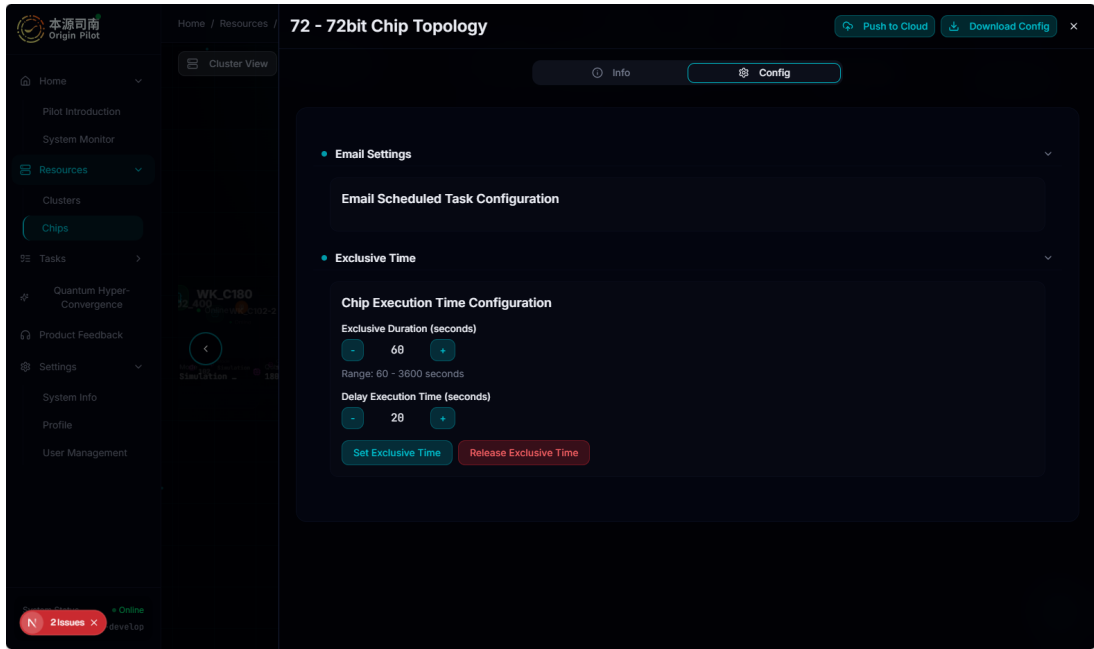


Figure 11: Advanced chip settings and scheduling management Configuration Tab.

4.4.3 Cluster Management Module

Function Overview: The Cluster Management and Details module provides a unified interface for monitoring and managing all compute clusters connected to PilotOS. Users can view cluster overviews, config data, node info, and task queue states in a single integrated view.

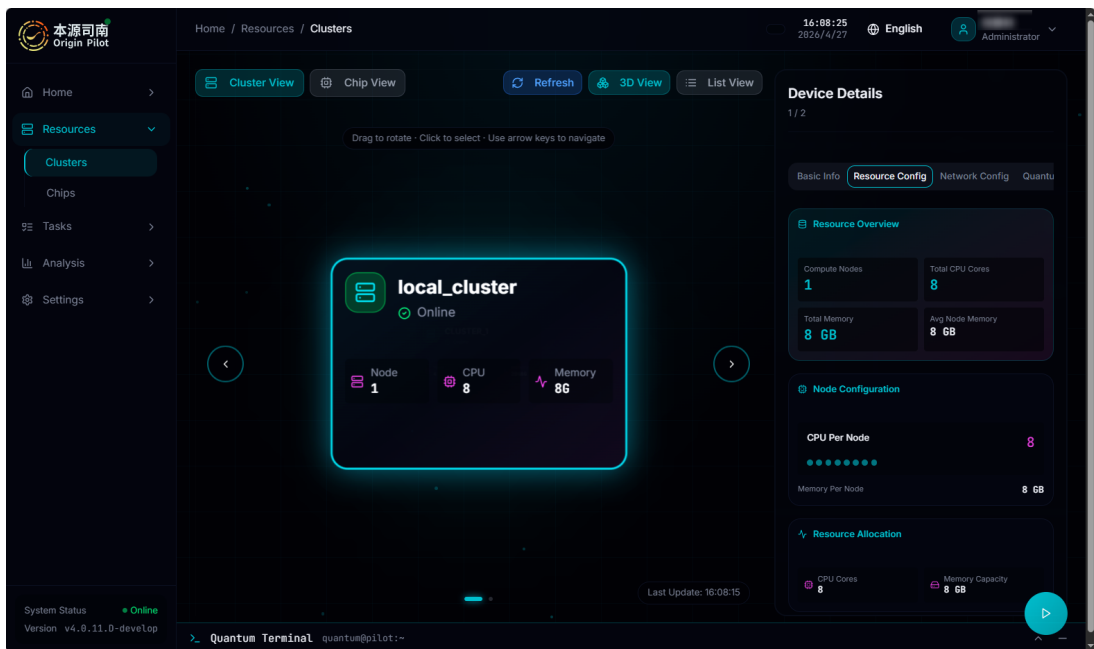


Figure 12: Cluster Management Overview Interface

Cluster Management As shown in Figure 12, each cluster is displayed as a summary card containing:

- **Cluster Name and Status:** Top left shows the cluster name, top right shows the master node IP address (if available).
- **Scheduling Status:** Online / Offline.
- **Configuration Summary:** Number of nodes/CPU, memory size.

4.4.4 Upgrade PilotOS Functional Module

The Pilot upgrade function is used for system version updates, supporting manual system upgrades.

Upgrade Entry: Users can access the upgrade process through the following methods (restricted to administrator accounts only):

- **Method 1:** Navigate to **Settings** → **System Info** in the sidebar, then click the **Check for Updates** button on the System Information page (see Figure 13).
- **Method 2:** On the System Information page, click the current version number area to directly trigger the upgrade process.



Figure 13: System Info

Upgrade Process: The upgrade process consists of the following steps:

1. Navigate to the System Information page (Sidebar → Settings → System Info), which displays the current version number and current version features (see Figure 13).
2. Click the **Check for Updates** button. The system displays the upgrade method selection dialog (see Figure 14):

- **Auto Upgrade:** The system automatically downloads and performs the upgrade (currently unavailable, the option is grayed out).
 - **Manual Upgrade:** Upload or place the upgrade package and then execute the upgrade (recommended).
3. After selecting **Manual Upgrade**, the system displays the upgrade guide dialog (see Figure 15). The dialog shows the upgrade package placement path and a drag-and-drop upload area. Users must place the upgrade package (`.tar.gz` / `.zip`) in the server directory: `/opt/PilotOS-Upgrade`.
 4. Click the **Placement Complete, Start Verification** button. The system automatically scans the specified directory for upgrade packages.
 5. Based on the scan results, the system proceeds as follows:
 - If only a **single package** exists, the system automatically selects it and proceeds directly to verification.
 - If **multiple packages** exist, the system displays the package selection dialog. Users can view each package's file name, version number, build information, package size, and upload time, then select the target package. The system sorts packages by version number and build number in descending order, defaulting to the latest version.
 6. The system automatically verifies the integrity and security of the selected upgrade package. A verification progress indicator is displayed during the process.
 7. After verification passes, the system displays a confirmation dialog showing the new version number and upgrade notes. Click **OK** to confirm and start the upgrade.
 8. The system enters the upgrade state and displays a full-screen upgrade progress overlay, showing real-time progress percentage and current operation status (e.g., notifying upgrade service, extracting and replacing system files, restarting Pilot service, etc.).
 9. After the upgrade is complete, the system automatically refreshes the page and loads the new version.

Notes:

- Do not close the browser or refresh the page during the upgrade process.
- System services will be temporarily interrupted during the upgrade.
- It is recommended to perform the upgrade operation during system idle times.
- Please ensure no critical tasks are running before initiating the upgrade.
- If the upgrade fails, please contact operations personnel promptly.
- Forcibly interrupting the upgrade may lead to system abnormalities or failure to boot.

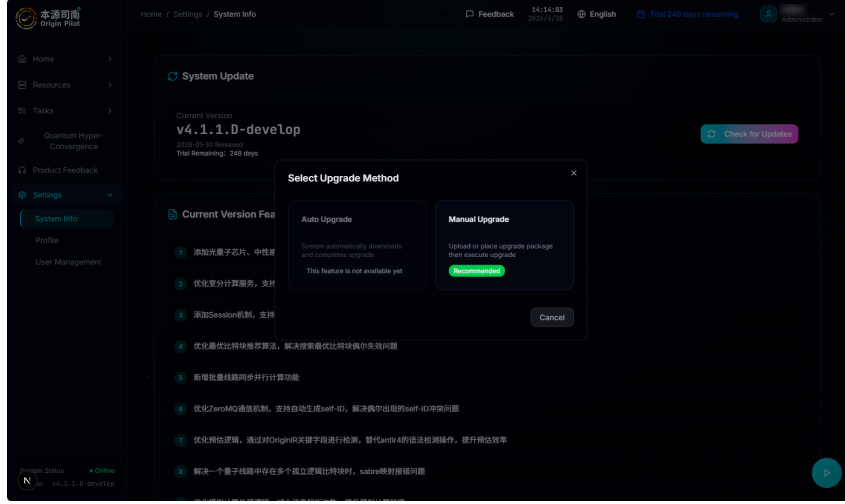


Figure 14: Upgrade Method Selection Dialog

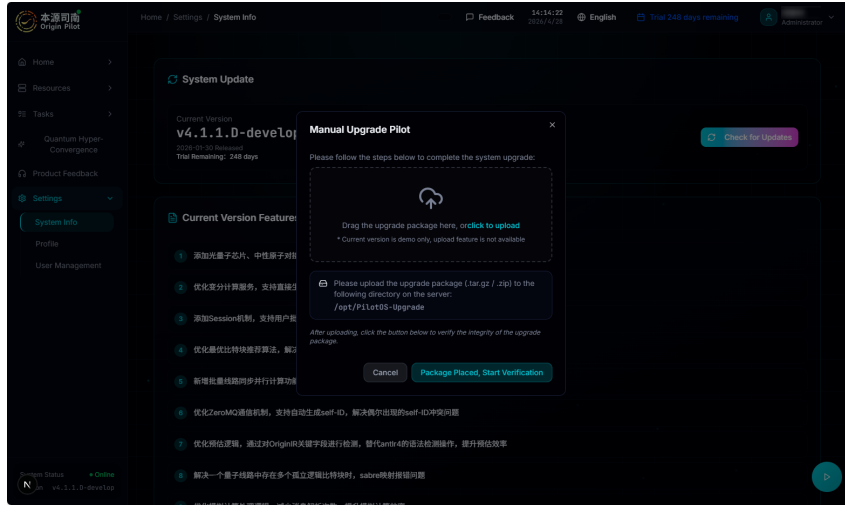


Figure 15: Upgrade Guide Dialog

4.5 Task Management

4.5.1 Task Submission Page

Function Overview: The Task Submission page is used to configure, submit, and view quantum compute tasks on the PilotOS platform, supporting real quantum chips and simulator tasks. Users can choose the compute type, select backends, configure parameters, upload or edit quantum programs (QProg), and view execution results.

The page simultaneously supports querying historical tasks via Task ID. When a matching task is found, its parameters, QProg, and results (if available) are automatically repopulated into the form, enabling task reproduction or resubmission.

Page Entry:

- Task Management → Task Submission
- Enter from *My Tasks* or *Task Details* pages to restore task info

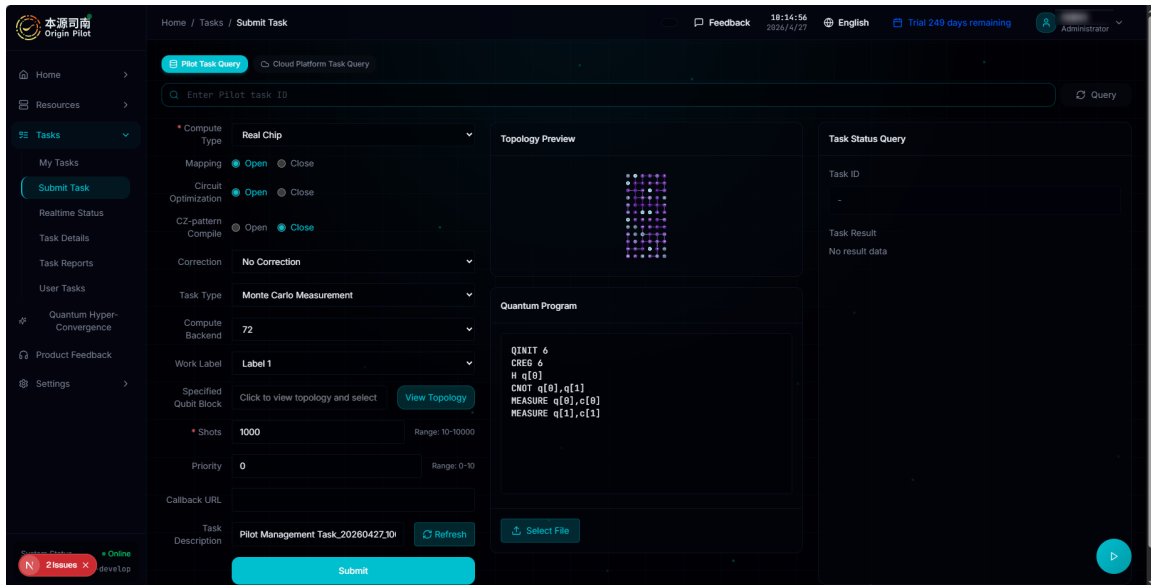


Figure 16: Task Submission Page Overview.

Page Structure:

As shown in Figure 16, the Task Submission page consists of four main functional areas:

- **Task Query & Restore Area (Top):** Used to query historical tasks and restore their parameters.
 - **Task ID Input Box:** Enter an existing Task ID
 - **Query Button:** Query the task and automatically restore parameters, QProg, and results (if available)
- **Task Configuration Area (Left):** Used to configure task parameters and select compute backends.
 - **Compute Type Selection:**
 - * Full Amplitude Simulation
 - * Noise Simulation
 - * Partial Amplitude Simulation
 - * Single Amplitude Simulation
 - * MPS Simulation
 - * Real Quantum Chip
 - **Backend Selection:**

- * Real Chip Task: Select **Chip ID**
- * Simulator Task: Select **Cluster ID**
- **General Parameters:**
 - * Measurement Type
 - * Number of Shots
 - * Target Qubits
 - * Specified Qubit Block (Select via topology dialog)
 - * Mapping Options [cite: 40]
 - * Noise Model
- **Chip Topology Dialog (Specified Qubit Block):** When configuring a specified qubit block, you can open the chip topology dialog to view the connection map, and select or deselect physical qubit blocks, as shown in Figure 17.

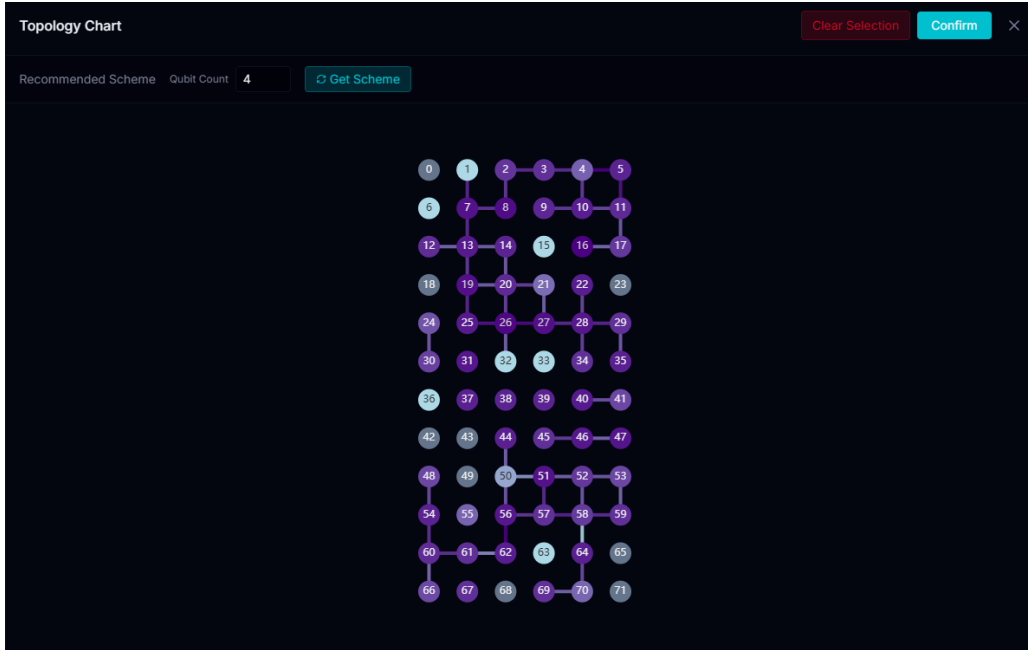


Figure 17: Chip Topology Dialog for Qubit Block Selection.

- **Advanced Settings (Click to expand):**
 - * Point Label
 - * Task Priority
 - * Callback Address
- **Submit Button:** Submits the configured task to the PilotOS system service.
- **Quantum Program Area (Right):** Used to input or upload QProg.
 - **Text Editor:** Direct pasting of QProg is allowed

- **File Upload:** Supports `.txt` files
- **Task Results Area (Bottom):**

After querying or submitting a task, the following info is displayed:

 - Task ID
 - Error Info (If any)
 - Pre-compilation / Post-compilation Instructions
 - Task Results

For tasks supporting visualization, results can be displayed as a bar chart.

Operation Flow:

1. **(Optional) Query and Restore Historical Task** Enter a Task ID and click "Query", the system automatically restores parameters, QProg, and task results (if available).
2. **Real Quantum Chip Task**
 - Select Compute Type: Real Quantum Chip
 - Select Chip ID
 - (Optional) Select physical qubit blocks via topology dialog [cite: 41]
 - Configure measurement type, number of shots, and other parameters
 - Enable mapping to avoid logical and physical qubit conflicts
 - (Optional) Configure advanced settings
 - Input or upload QProg
 - Submit task
3. **Cluster Simulation Task**
 - Select Simulator Type
 - Select Cluster ID
 - Fill in simulator related parameters
 - Input or upload QProg
 - (Optional) Set priority or description
 - Submit task

4.5.2 My Tasks Page

Function Overview:

The My Tasks page is used to display and manage the quantum compute tasks submitted by the current user within the PilotOS system service. It can be accessed via the **My Tasks** sub-menu under Task Management. On this page, users can view task lists,

search history records, filter tasks by multiple conditions, and jump to the task details page or task submission parameter page.

Page Layout:

As shown in Figure 18, the My Tasks page primarily consists of the following functional modules:

- **Task Query Module (Top):** Provides multi-condition search functionality, supporting the following filter options:
 - Compute Type (e.g., Full Amplitude Simulation, Noise Simulation, Partial Amplitude Simulation, etc.); [cite: 42]
 - Backend Type (Quantum Chip or Cluster);
 - Time Range (Task creation time range);
 - Task Description (Keyword search).

The page also provides **Search**, **Reset**, and **Add New Task** buttons for quick operations.

- **Task List Module (Main Area):** Displays task information in tabular format, including:
 - **Task ID:** Clickable to jump to the Task Details page;
 - **Task Description**
 - **Task Type**
 - **Compute Backend**
 - **Task Status:** Such as Running, Completed, Failed; [cite: 43]
 - **Start Time, End Time, and Total Duration**
 - **Actions:** Click **View Parameters** to jump to the Task Submission page to review submission configurations.

4.5.3 Task Details Page

Function Overview:

The Task Details page is used to view detailed info and timestamp data of a single quantum compute task. It can be opened via **Task Management** → **Task Details**, or by clicking the Task ID in the My Tasks list. Users can query tasks by Task ID and view the timestamps of each processing stage, task status, and calculation results (as shown in Figure 19).

Page Layout:

The Task Details page includes the following functional modules:

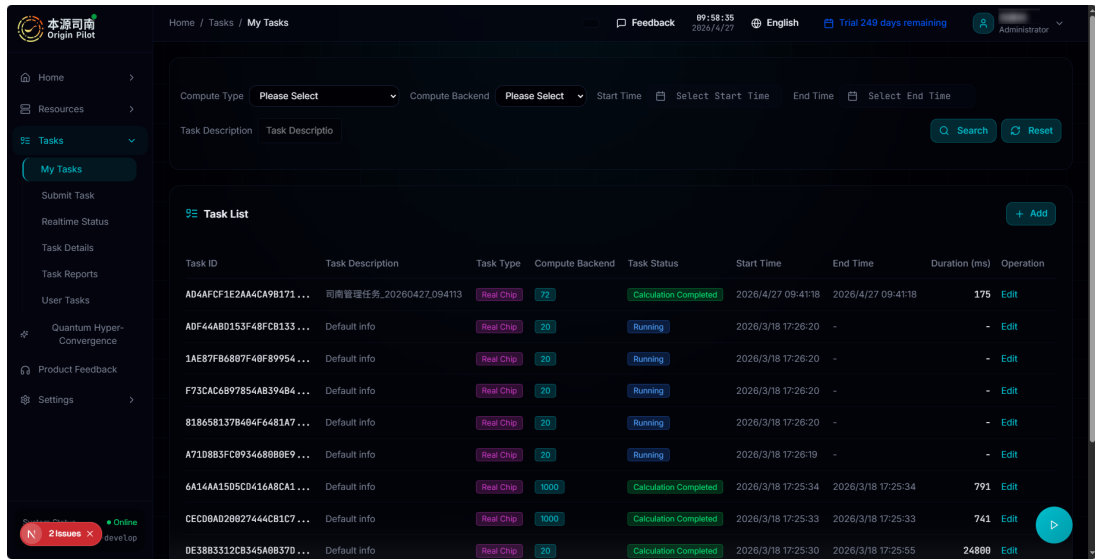


Figure 18: My Tasks Page, showing task list, search filters, and links to task details and parameters.

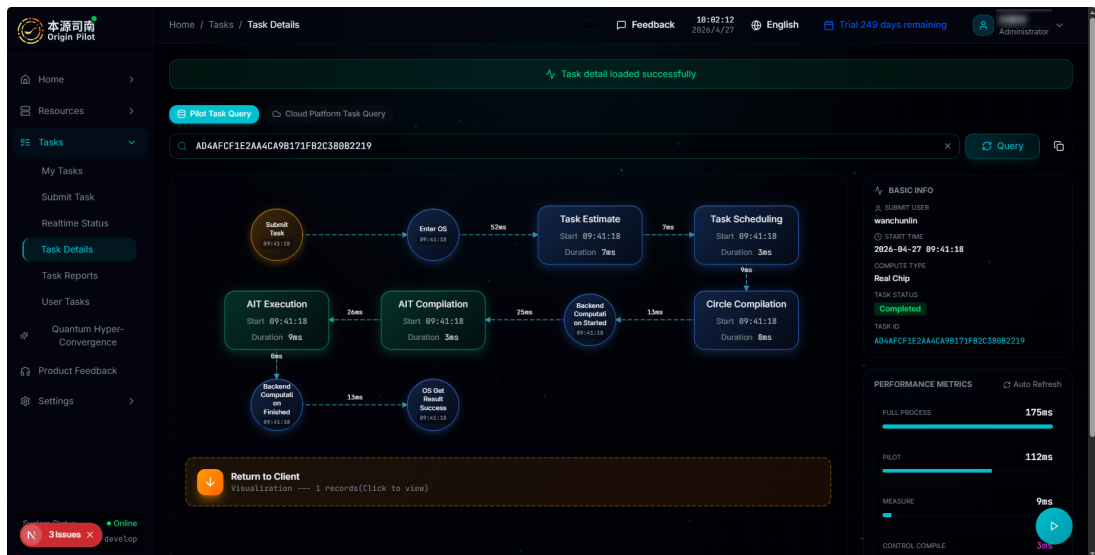


Figure 19: Task Details Page Overview: Supports querying by Task ID, displaying task timestamps, module execution times, and compute results.

- **Task Query Module (Top):** Enter a Task ID and click **Search** to retrieve task details and timestamp data.
- **Task Timestamp Visualization Module (Left):** Displays task execution time nodes and status across modules as a flow chart. Shows an empty state when no task data is available.
- **Task Information Module (Right):** Displays detailed task information via cards, including:
 - Task [cite: 44] ID (Clickable to jump to Task Submission page to view parameters);
 - Error Info (If any);
 - Submitter;
 - Submission Time;
 - Compute Type (Quantum Chip or Cluster Computing);
 - Task Status;
 - Total Processing Time, Pilot Total Time, and phase durations (Compilation, Queuing, Measurement, Post-processing, etc.);
 - Task Results.

4.5.4 Real-Time Task Status Page

Function Overview:

The Real-Time Task Status page is used to display tasks currently queued and executing in the system. Enter this page via **Task Management** → **Real-Time Task Status**. Users can monitor active tasks, view the latest update times, check task details, and search by Task ID, as shown in Figure 20.

Page Layout: [cite: 45]

- **Task Query Module:**
 - **Task ID Query:** Input Task ID to locate a specific task;
 - **Auto-Refresh Control:**
 - * *Enable Refresh:* Continuously update task status;
 - * *Stop Refresh:* Pause auto-updating.
 - **Total Tasks:** Displays total number of queued tasks in the system.
- **Task List Table:** [cite: 46] Displays tasks in a sortable table, columns include:
 - **Task ID** (Click to jump to Task Details page);

- Backend;
- Submitter;
- Submission Time;
- Task Status;
- Status Code.

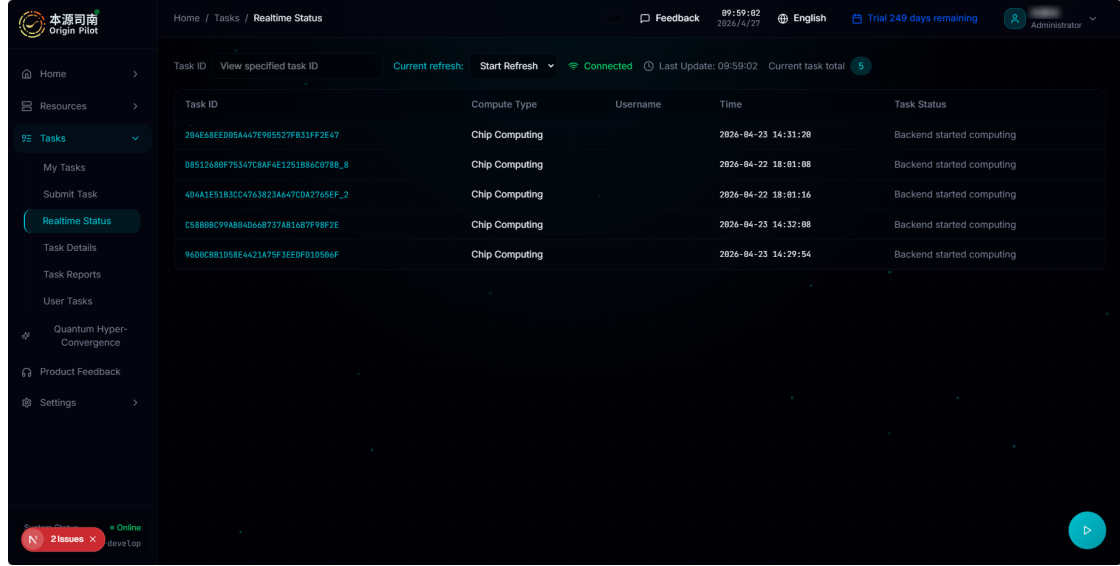


Figure 20: Real-Time Task Status Page, showing queued and executing tasks, Task ID querying, and auto-refresh controls.

4.5.5 User Task Management Page

Function Overview:

The User Task Management page is used to display and manage queued tasks submitted by users. Users can view their own tasks in backend queues, and cancel their own tasks. Administrators can manage all users' tasks.

Access Path: **Task Management** → **User Task Management**. Page overview in Figure 21.

Page Layout:

- **Top Action Area:**
 - **Batch Cancel:** Select multiple tasks and click **Cancel Task**. [cite: 47]
 - **Queue Size:** Displays current total task queue count.
 - **Account Name Filter:** Admins can filter tasks by submitter account.
- **Task List Table:** Displays queued tasks, columns include:

- **Select** (Users can only select their own submitted tasks);
- **Task ID**;
- **Compute Type**;
- **Task Status**;
- **Submitter**; [cite: 48]
- **Submission Time**;
- **Actions**: Set Priority / Cancel Task.

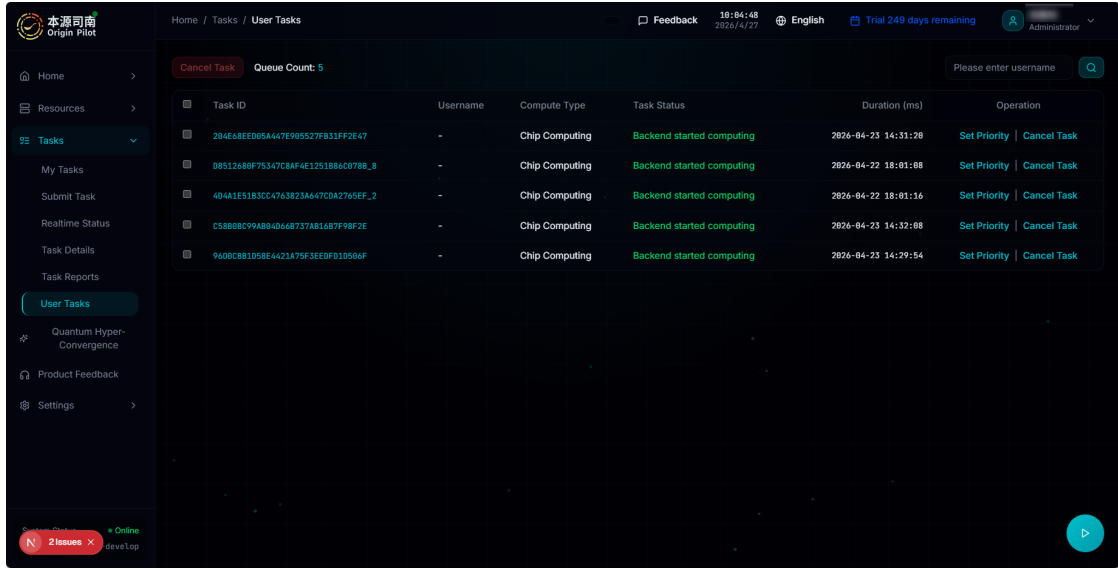


Figure 21: User Task Management Page

4.5.6 Task Report Page

Function Overview:

The Task Report page is used to query, filter, and export historical task data. Users can filter by time range, account name, compute type, and task status, supporting customized display columns. The page supports task list displays, field selection, Task ID copying, and CSV exporting.

Access Path: **Task Management** → **Task Report**. Page overview in Figure 22.

Page Layout:

- **Query Conditions**: Configure report filters:
 - **Time Range (Required)**: Use datetime selectors to pick start and end times.
 - **Account Name**: Filter tasks by submitter.
 - **Compute Type**: Includes simulator and real quantum chip.

- **Task Status:** [cite: 49]
 - * Computing
 - * Completed
 - * Failed
 - * Queuing
 - * Cancelled
- **Actions:** [cite: 50]
 - * **Search:** Load filtered tasks;
 - * **Download:** Export results in CSV format.
- **Column Selection:** Select displayable fields, including:
 - Task ID
 - Account Name
 - Task Status [cite: 51]
 - QProg
 - Backend ID
 - Task Type
 - Config Info
 - Task Results
 - QST Results / QST Fidelity
 - Various Time Metrics
 - Start Time / End Time [cite: 52]
 - Error Code / Error Info
 - SN / MC Timestamps
 - Task Description
- **Task List Table:** Displays paginated filtered tasks:
 - **Task ID:** Fully displayed, clickable to jump to Task Details page, and provides a quick copy button;
 - **Task Status:** Displayed using colored tags;
 - **Dynamic Columns:** Table columns dynamically adjust based on user selection.

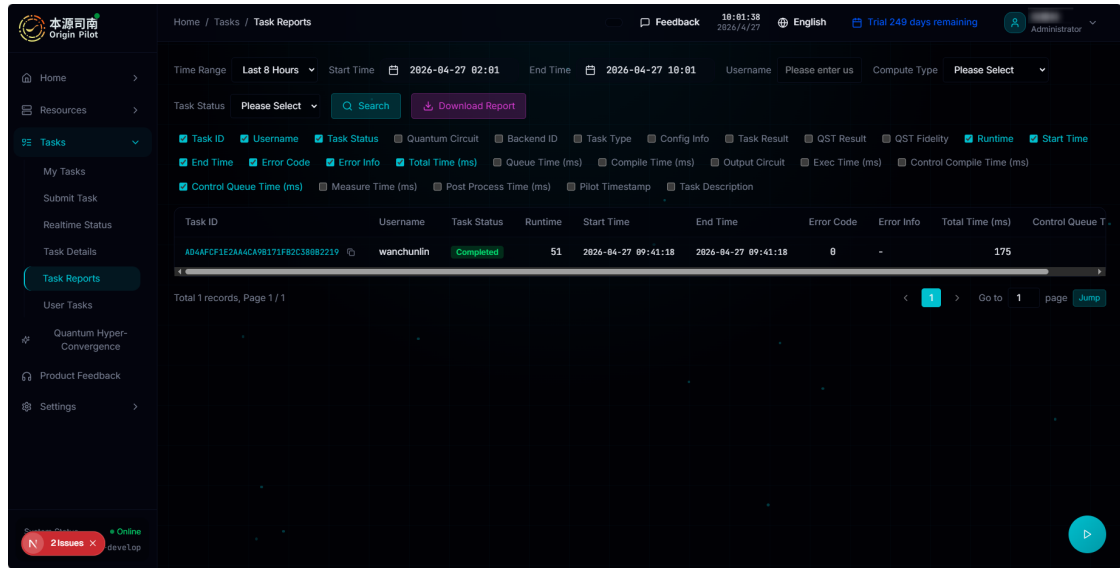


Figure 22: Task Report Page

5 API Reference

Below is a detailed introduction to PilotOS APIs and code examples.

- RESTful [cite: 53] API Interface List (GET / POST / PUT / DELETE)
- Request URLs, Parameters, and Examples
- Response Formats (JSON)
- Error Code Directory
- SDK Usage Examples (Python / cURL)

5.1 PilotOS Login

URL: <https://xx.com/management/pilotosmachineLogin>

The login function is a prerequisite for using PilotOS OpenAPI, used for identity verification. The Token acquired after login allows the user to execute subsequent operations (e.g., submitting tasks). When logging in, the user's API Key must be used for identity verification.

Request Method: Post

Required Parameters:

- apiKey(str): The "digital ID" or "dedicated key" used by PilotOS.

Return Values:

- errCode(uint): Error code.
- errInfo(str): Error information.
- token(str): The user identity identifier for this login.

Request Example:

```
1 {  
2     "apiKey": "D13A8E5ADxxxxxxxxF36A4B940D18CEF"  
3 }
```

Response Example:

```
1 {  
2     "errCode": 0,  
3     "errInfo": "",  
4     "token": "c3f7fe35-xxxx-xxxx-a178-070cc311d03b"  
5 }
```

5.2 Submit Quantum Chip Computing Task

URL: <https://xxx.com/task/realQuantum/run>

Submit a calculation task to a real physical quantum chip. Note: Tasks submitted to the quantum chip **must** contain Measurement operations.

Request Method: Post

Required Parameters:

- QProg(str): Quantum circuit.
- TaskType(int): Task type.
 - 1 Undefined
 - 0 [cite: 54] Monte Carlo Measurement
 - 2 Quantum State Tomography (QST)
 - 5 Expectation Value
 - 10 Distributed

Optional Parameters:

- Priority(uint): Priority. 0 means normal task (default value), larger values indicate higher priority.
- ChipID(str): Used Chip ID. Currently provides 72 and 102 qubit chips, default is 72.
- Configuration(dict): Config items.
 - shot(uint): Number of measurements, default 1000, range 10-20000.
 - circuitOptimization(bool): Whether to enable circuit optimization. True enables (default), false disables.

Return Values:

- taskState(str): Task status.

0	Not operated
1	Sent
2	Running
3	Completed (Control system returned results)
5	Compilation complete, not sent to control system
6	Compilation complete, sent to control system
7	Control system calculation error
8	Run time exceeded
9	Compilation system started execution
10	Results returning to compilation system
11	Compilation system received task and started calculation
12	Compilation system returned results
22	Compilation system handling parallel tasks
30	Get parallel calculation results, sent to compilation system for processing
31	Received task, ready to process
32	Processing parallel results
33	Parallel results processing complete
34	Parallel results processing error

- taskId(str): Task ID of the submitted task.

Request Examples:

1. [cite: 55] Normal Measurement Task:

```

1 {
2   "token": "c3f7fe35-xxxx-xxxx-a178-070cc311d03b",
3   "QMachineType": 5,
4   "QProg": [
5     "QINIT 6\nCREG 6\nH q[0]\nCNOT q[0],q[1]\nMEASURE q[0],c
        [0]\nMEASURE q[1],c[1]"
6   ],
7   "ChipID": "72",
8   "TaskType": 0,
9   "Priority": 0,
10  "taskDescribe": "default",
11  "Configuration": {
12    "shot": 1000,
13    "circuitOptimization": True,
14    "use_cz_pattern": False,

```

```

15     "amendFlag": 0,
16     "mappingFlag": True,
17     "specified_block": [],
18     "PointLabel": 1
19 }
20 }

```

2. [cite: 56] Quantum State Tomography Task:

```

1 {
2     "token": "c3f7fe35-xxxx-xxxx-a178-070cc311d03b",
3     "QMachineType": 5,
4     "QProg": [
5         "QINIT 6\nCREG 6\nH q[0]\nCNOT q[0],q[1]\nMEASURE q[0],c
          [0]\nMEASURE q[1],c[1]"
6     ],
7     "ChipID": "72",
8     "TaskType": 2,
9     "Priority": 0,
10    "Configuration": {
11        "shot": 1000,
12        "circuitOptimization": True,
13        "use_cz_pattern": False,
14        "amendFlag": 0,
15        "mappingFlag": True,
16        "specified_block": [],
17        "PointLabel": 1
18    }
19 }

```

3. [cite: 57] Distributed Task:

```

1 {
2     "QMachineType": 5,
3     "ChipID": "72",
4     "TaskType": 10,
5     "token": "c3f7fe35-xxxx-xxxx-a178-070cc311d03b",
6     "QProg": [
7         "QINIT 6\nCREG 6\nRY(q[0], 0.162461319481123)\nRY(q[1],
          1.13411443752956)\nRY(q[2], 0.349219035305167)\nCNOT(q
          [0], q[1])\nRY(q[0], -1.30314609114098)\nRY(q[1],
          -0.000403011347911202)\nCNOT(q[1], q[2])\nRY(q[1],
          -0.0668867624864373)\nRY(q[2], 0.751632608892654)\nCNOT(

```

```

      q[2], q[0])\nRY(q[2], 0.943957761048972) \nRY(q[0],
      0.877382733682492)\nCNOT(q[2], q[0])\nCNOT(q[1], q[2])\
      nCNOT(q[0], q[1])"
8   ],
9   "Configuration": {
10     "shot": 1000,
11     "mappingFlag": true,
12     "construction_type" : 0,
13     "max_supported_qubits" : 2
14 [cite: 58]   }
15 }

```

4. Expectation Calculation Task:

```

1 {
2   "token": "c3f7fe35-xxxx-xxxx-a178-070cc311d03b",
3   "QMachineType": 5,
4   "QProg": ["QINIT 6\nCREG 6\nH q[0]\nCNOT q[0],q[1]"],
5   "ChipID": "72",
6   "TaskType": 5,
7   "Configuration": {
8     "shot": 1000,
9     "circuitOptimization": True,
10    "hamiltonian": "\"\": -1.0534210769165204,\"Z0\":
      0.394844363355904,\"Z1\": -0.39484436335590417,\"Z0 Z1
      \": -0.01124615715082114,\"X0 X1\": 0.18121046201519694"
11  }
12 }

```

Response Example:

```

1 {
2   "taskId": "098D12FE69184646BF16DDF53930C569",
3   "taskState": 1
4 }

```

5.3 Query Task

URL: <https://xxx.com/management/query/taskinfo>

Query a previously submitted task, returning its execution status and related information. If the task is completed, it also returns the task result.

Request Method: Post

Required Parameters:

- taskid(str): Task ID
- token(str): Identity [cite: 59] Token

Return Values:

- taskId(str): Task ID
- TaskType(int): Task type
- taskState(uint): Task status
- qProgLength(uint): Number of quantum circuits
- qProg(arr[str]): Quantum circuit details
- taskResult(str): Quantum task execution result

Request Example:

```
1 {
2     "taskId": "098D12FE69184646BF16DDF53930C569",
3     "token": "c3f7fe35-xxxx-xxxx-a178-070cc311d03b"
4 }
```

Response Example:

```
1 {
2     "taskId": "098D12FE69184646BF16DDF53930C569",
3     "TaskType": 1,
4     "taskState": 3,
5     "qProgLength": 1,
6     "without_compensate_prog": "",
7     "qProg": [
8         "QINIT 4\nCREG 4\nH q[0]\nMEASURE q[0],c[0]"
9     ],
10    "taskResult": "[{"key":["0x0","0x1"],"value":[0.52,
11        0.48]}]"
11 }
```

5.4 Get Queue Information

URL: <https://xxx.com/system/queryBackendQueueTask>

Get PilotOS chip backend task queue information.

Request Method: Post

Required Parameters:

- backend(str): Chip ID.

- `maxQueryTaskSize(uint)`: Maximum number of queried tasks.

Return Values:

- `msg_type(str)`: Message type.
- `backend(uint)`: Backend ID.
- `is_ok(bool)`: [cite: 60] Whether the query was successful.
- `queue_task_size(uint)`: Total number of tasks found.
- `myTask(arr[str])`: Collection of retrieved task IDs.
- `userNames(arr[str])`: Collection of user names corresponding to the tasks.

Request Example:

```

1 {
2     "backend": "72",
3     "maxQueryTaskSize": 500
4 }
```

Response Example:

```

1 {
2     "msg_type": "schdule_queue_resp",
3     "backend": "72",
4     "is_ok": true,
5     "queue_task_size": 2,
6     "myTask": ["FCECF3B04053402895C2485E67FAEB80", "
7         B859CD543212488DB8582B45E940CFC9"],
8     "userNames": ["user1", "user2"]
9 }
```

5.5 Get Chip Information

URL: <https://xxx.com/management/getchipconfig>

Get detailed configuration info of the chip.

Request Method: Post

Required Parameters:

- `ChipID(str)`: Chip ID.
- `Label(uint)`: Chips under different labels have different characteristics (like fidelity).

Return Values:

- `QubitParams(json obj)`: Parameters like fidelity, coherence time for each qubit.
- `NoiseModel(json obj)`: Chip noise model under real environment.

- `CompensateAngle(json obj)`: Crosstalk compensation for CZ gates between pairs of qubits.
- `AvailableQubits(json obj)`: Available qubits on the chip.

Request Example:

```

1 {
2     "ChipID": "72",
3     "Label": 1
4 }
```

Response Example:

```

1 {
2     "QubitParams":
3     [
4         [cite: 61] {
5             "qubitId": "45",
6             "FidelityMat": [[0.9556, 0.1218], [0.0444, 0.8782]],
7             ...
8         },
9         ...
10    ],
11    "NoiseModel": [
12        {"modelType": "noise_model_single"},
13        ...
14    ],
15    "CompensateAngle":
16    {
17        "CZ(45, 46)": {"45": 2.367866260192345...},
18        [cite: 62]...
19    },
20    "AvailableQubits": [45, 46, 48, 52, 53, 54],
21    ...
22 }
```

5.6 Submit Cluster Simulation Compute Task

URL: <https://xx.com/task/run>

Submit a compute task to the cluster for simulation.

Request Method: Post

Parameters Description:

Param Desc	Param Name	Type	Remarks	Nullable
Quantum Program	QProg	String		No
QProg Length	QProgLength	uint		Yes
Config Info	Configuration	json	Config for different task types	No
Task Type	TaskType	uint	Refer to task type definitions	Yes
VM Type	QMachineType	String	0:Full Amp, 1:Noise, 2:Partial Amp, 3:Single Amp, 4:MPS	No
VIP Type	Priority/VIP	uint/string	0 is normal task, higher values mean higher priority	Yes
Cluster ID	ClusterID	string	Assigned by system if empty	Yes [cite: 63]
Callback Address	callbackAddr	String		Yes
Unique ID	Token	string	User Identifier	Yes

5.6.1 Configuration Info Description and Request Examples

(1) Full Amplitude Virtual Machine (QMachineType: 0)

Configuration Parameters:

- `measure_type` (uint): 1-Measure; [cite: 64] 2-PMeasure; 3-Hamiltonian (Not empty).
- `shot` (uint): Measurement times (Not empty).
- `qubits` (uint array): Must be specified for PMeasure and Hamiltonian.
- `hamiltonian` (array/string): Hamiltonian parameters (Not empty when `measure_type` is 3).
- `hamiltonian_param` (double): Hamiltonian coefficient (when in array format).
- `pauli_type` (str array): Pauli operator type (when in array format).
- `pauli_param` (double array): Target qubits of the operator (when in array format).

Request Example (Measure):

```

1 {
2   "QMachineType": "0",
3   "QProg": "QINIT 4\r\nCREG 4\r\nnTOFFOLI q[3],q[0],q[1]\r\nMEASURE q
         [1],c[1]",
4   "QProgLength": 58,
5   "Configuration": {
6     "measure_type": 1,
7     "shot": 1000
8   }
9 }
```

Request Example (PMeasure):

```

1 {
2   "QMachineType": "0",
3   "QProg": "QINIT 4\r\nCREG 4\r\nnH q[0]\r\nnCNOT q[0],q[1]...",
4   "QProgLength": 71,
5   "Configuration": {
```



```

6     "measure_type":2,
7     "qubits":[0,1,2,3]
8 }
9 }

```

Request Example (Hamiltonian):

```

1 {
2   [cite: 65] "QMachineType":"0",
3   "Configuration":{
4     "qubits":[0,1,2,3,4,5],
5     "hamiltonian":[
6       {"pauli_type": ["X", "Y"], "pauli_param": [0, 1], "
7         hamiltonian_param": 0.2},
8       {"pauli_type": ["Y", "X"], "pauli_param": [2, 3], "
9         hamiltonian_param": 0.8}
10    ],
11    "measure_type":3
12 }

```

(2) Partial Amplitude Virtual Machine (QMachineType: 2)

Configuration Parameters:

- Amplitude (string array): Decimal index array (Not empty).

Request Example:

```

1 {
2   "QMachineType":"2",
3   "Configuration":{
4     "Amplitude":["0"]
5   }
6 }

```

(3) Single Amplitude Virtual Machine (QMachineType: 3)

Configuration Parameters:

- Amplitude (string): Decimal index (Not empty).

Request Example:

```

1 {
2   "QMachineType":"3",
3   "Configuration":{
4     "Amplitude":"0"
5   }
6 }

```

```

5   }
6 }

```

(4) Noise Virtual Machine (QMachineType: 1)

Configuration Parameters:

cite: 66 measure_type (uint): Must be 1.

- shot (uint): Measurement times.
- noisemodel.single_gate (array): Single gate operation data [Type str, Param double].
- noisemodel.double_gate (array): Double gate operation data [Type str, Param double].
- mode (uint): Noise model type.

Request Example:

```

1 {
2   "QMachineType": "1",
3   "Configuration": {
4     "mode": 1,
5     "measure_type": 1,
6     "noisemodel": {
7       "single_gate": ["DAMPING_KRAUS_OPERATOR", 0.100000],
8       "double_gate": ["DAMPING_KRAUS_OPERATOR", 0.200000]
9     },
10    "shot": 1000
11  }
12 }

```

(5) MPS Virtual Machine (QMachineType: 4)

Configuration Parameters:

- measure_type (uint): 1-Measure; [cite: 68] 2-PMeasure.
- shot (uint): Measurement times.
- qubits (int array): Not empty for PMeasure.

Return Parameters Description:

- taskState (string): Task status.
- taskId (string): Task ID.

5.7 Cluster Simulation Task Status Query

URL: <https://xx.com//task/query>

Request Method: Post

Parameters:

- taskId (String): Task ID (Not empty).

Request Example:

```
1 {"taskId": "5EB2161414632304351f8a9C5BBE124ef1b31d71ced321fA92"}
```

5.8 Get Task Queuing Info for Specified Backend

URL: <https://xx.com/system/queryBackendQueueTask>

Request Method: Post

Parameters:

- backend (string): Chip ID (Not empty).
- maxQueryTaskSize (uint32): Maximum returned task count (Not empty).
- token (string): Unique identifier (Nullable).

Return Example:

```
1 {
2   "msg_type": "schdule_queue_resp",
3   "backend": "72",
4   "is_ok": true,
5   "queue_task_size": 2,
6   "queue_task": ["task_id_1", "task_id_2"],
7   "myTask": ["FCECF3B04...", "B859CD5..."],
8   "userNames": ["yebin", "yebin"]
9 }
```

5.9 Get Chip Calibration Time

URL: <http://xx.com/system/queryChipUpdateTime>

Request Method: Post

Parameters:

- backend (string): Backend ID (Not empty).

Request Example:

```
1 {"backend": "72"}
```

Return Example:

```

1 {
2     "backend": "72",
3     "chipLastUpdateTime": {
4 [cite: 69]         "qubit": [43, 45...],
5         "timeStamp": [1687243363189...]
6     },
7     "errCode": 0
8 }

```

5.10 Pause Chip Scheduling Service

URL: <http://xx.com/system/pauseChipSchedule>

Request Method: Post

Parameters:

- ChipID (string): Chip name (Not empty).
- isPause (bool): true enables scheduling, false pauses scheduling (Nullable, default true).

Request Example:

```

1 {
2     "isPause": false,
3     "ChipID": "72"
4 }

```

Return Example:

```

1 {
2     "scheduleState": false,
3     "errCode": ,
4     "errInfo": ""
5 }

```

5.11 Get Chip Schedule State

URL: <http://xx.com/system/queryChipScheduleState>

Request Method: Post

Parameters:

- ChipID (string array): Array of Chip IDs (Not empty).

Request Example:

```

1 {
2     "ChipID":["1","72"]
3 }

```

Return Example:

```

1 {
2     "ScheduleState" :[
3         { "chipID":"1", "status":true },
4         [cite: 70] { "chipID":"72", "status":true }
5     ],
6     "errCode":0,
7     "errInfo":""
8 }

```

5.12 Manual M3 Sampling

URL: <http://xx.com/system/manualSampleM3>

Request Method: Post

Parameters:

- Token (String): User Token.
- ChipID (string): Chip ID.
- PointLabel (Uint): Point label.

Request Example:

```

1 {
2     "Token":"3A9D2FD3ECCB4B0C",
3     "ChipID":"72",
4     "PointLabel":1
5 }

```

Return Example:

```

1 {
2     "errCode":0,
3     "errInfo":""
4 }

```

5.13 Session Mechanism Task Management

Users can apply for a session and submit tasks within the session unit, granting them higher priority.

5.13.1 Request Session

URL: <http://xx.com//session/apply>

Parameters:

- LifeTime (int): Session valid time (minutes).
- Token (String): User Token.

Request Example:

```
1 {  
2     "LifeTime" : 30,  
3     "Token" : "D13A8E5AD804478FBF36A4B940D18CEF"  
4 }
```

Return Example:

```
1 { "SessionId" : [cite: 71] "6D94C74A...", "errCode": 0 }
```

5.13.2 Submit Task in Session

URL: http://xx.com//session/append_task

Parameters: Same as "Submit compute task to physical quantum chip", with additional parameter required:

- SessionId (string): Session ID (Not empty).

Request Example:

```
1 {  
2     "QMachineType": "5",  
3     "ChipID": "72",  
4     "TaskType": 3,  
5     "QProg": ["QINIT 4\r\nCREG 4\r\nH q[3]..."],  
6     "QProgLength": 2,  
7     "Configuration": {  
8         ...  
9     },  
10    "taskDescribe": "default",  
11    "SessionId": "6D94C74A73..."  
12 }
```

Return Example:

```
1 { "taskState": "1", "taskId": "6D94C74A73A544FD9C445D51E7436191_0" }
```

5.13.3 Close Session

URL: <http://xx.com//session/release>

Parameters:

- SessionId (string): Session ID.
- Token (string): Token.

Request Example:

```
1 {
2   [cite: 72]   "SessionId" : "6D94C74A73A544FD9C445D51E7436191",
3   "Token"     : "D13A8E5AD804478FBF36A4B940D18CEF"
4 }
```

Return Example:

```
1 {
2   "SessionId": "6D94C74A73A544FD9C445D51E7436191",
3   "isSuccess": false,
4   "errCode": 0,
5   "errInfo": "Task is running."
6 [cite: 73] }
```

5.14 Get Chip Best Qubit Blocks

URL: http://xx.com//management/query/best_qubit_blocks

Request Method: Post

Parameters:

- ChipID (string): Chip ID.
- Label (uint32): Label type.
- QubitNum (uint32): Qubit number.
- QubitBlockNum (uint32): Qubit block number.

Request Example:

```
1 {
2   "ChipID" : "72",
3   "Label"  : 1,
4   "QubitNum" : 4,
5   "QubitBlockNum" : 1
6 }
```

Return Example:

```
1 {  
2   "ChipID": "72",  
3   "QubitBlocks": [[1, 2, 3, 4]],  
4   "errCode": 0  
5 }
```

5.15 Get Task Priority

URL: http://xx.com///task/query_task_priority

Request Method: Post

Parameters:

- TaskID (string): Task ID.
- Token (string): User identifier.

Request Example:

```
1 {  
2   "TaskID" : "ABC",  
3   "Token" : "asedsdfsdg",  
4 }
```

Return Example:

```
1 { "TaskID": "ABC", "Priority": 10 }
```


6 Message Push

The system pushes critical event messages to the configured addresses. Below is detailed information and push examples for each message type.

6.1 Chip Calibration Start Information

Push Address: <http://xxx:31733/v5/oqcs/task/updateChipOnlineStatus.json>

Parameters Description: (If there is no calibration qubit information or CZ qubit pair information, it will not be pushed)

- msg_type (string): Fixed as "chip_calibration_start".
- chip_id (string): Chip ID.
- calibration_qubits (string array): Calibration qubits.
- calibration_couplers (string array): Couplers.
- calibration_pairs (string array): CZ qubit pairs.
- calibration_discriminators (string array): Discriminators.

Push Example:

```
1 {  
2   "msg_type": "chip_calibration_start",  
3   "chip_id": "72",  
4   "calibration_qubits": ["q0", "q1", "q2", "q3"],  
5   "calibration_couplers": ["c0-1", "c1-2", "c2-3"],  
6   "calibration_pairs": ["q0q1", "q2q3"],  
7   "calibration_discriminators": ["q0_01.bin"]  
8 }
```

6.2 Chip Calibration End Information

Push Address: <http://xxx:31733/v5/oqcs/task/updateCalculateResource.json>

Parameters Description:

- chipId (string): Chip ID.
- lastCalibrationTime (string): Last calibration timestamp.
- lastCalibrationBit (string array): Calibration qubit list.

cite: 75 lastCalibrationBitPair (string array): Calibration qubit pair list.

Push Example:

```
1 [  
2   {
```

```

3     "chipId": "72",
4     "lastCalibrationTime": "1731576994075",
5     "lastCalibrationBit": [],
6     "lastCalibrationBitPair": []
7 }
8 ]

```

6.3 Chip New Configuration Information

Push Address: <http://xxx:31733/v5/oqcs/task/updateChipConfig.json>

Parameters Description:

- msg_type (string): Fixed as "chip_update".
- chip_id (string): Chip ID.
- new_chip_config (json string): New chip configuration JSON string.

Push Example:

```

1 {
2   "msg_type": "chip_update",
3   "chip_id": "72",
4   "new_chip_config": "{
5     \"QuantumChipArch\": {
6       \"QubitCount\": 72,
7 [cite: 76]       \"HighFrequencyQubit\": [
8         0,
9         4,
10        9,
11        13,
12        16,
13        21,
14        24
15        [cite: 77] ]
16     }
17   }"
18 }

```

6.4 Chip Manual Maintenance Start Information

Push Address: <http://xxx:31733/v5/oqcs/task/updateChipOnlineStatus.json>

Parameters Description:

- msg_type (string): Fixed as "chip_manual_maintenance_start".
- chip_id (string): Chip ID.

Push Example:

```
1 {  
2   "msg_type": "chip_manual_maintenance_start",  
3   "chip_id": "72"  
4 }
```

6.5 Chip Manual Maintenance End Information

Push Address: <http://xxx:31733/v5/oqcs/task/updateChipOnlineStatus.json>

Parameters Description:

- msg_type (string): Fixed as "chip_manual_maintenance_end".
- chip_id (string): Chip ID.

Push Example:

```
1 {  
2   "msg_type": "chip_manual_maintenance_end",  
3   "chip_id": "72"  
4 }
```

7 Type Definitions and Error Codes

7.1 Task Types

Type Code	Description
-1	Unknown task type
0	Monte Carlo measurement task
1	PMeasure task
2	QST task
3	QST density matrix calculation task
4	Fidelity calculation task
5	Expectation calculation task
8	Noise learning task
9	Readout error mitigation task
10	Quantum distributed computing task
11 [cite: 78]	Fidelity measurement task

7.2 Compute Backend Types

Type Code	Description
0	Full amplitude simulation
1	Noise simulation
2	Partial amplitude simulation
3	Single amplitude simulation
4	MPS simulation
5	Real chip computing

7.3 Task Status Types

Status Code	Description
0	Unknown status
1	System received calculation task
2	Being executed
3	Calculation successful
4	Calculation failed
5	Queuing
35	Task has been cancelled

7.4 Error Codes (ErrorCode)

Error Code	Description
0	No error
1	Database error
2	OriginIR syntax error
3	Json parsing error
4	Backend computing error
5	System task queue is full
6	Qubit count exceeds limit
7	[cite: 79] Unsupported backend type
8	Quantum program timing exceeded
9	Task type error
10	Virtual machine initialization failed
11	Compilation failed
12	Task estimation error
13	Metadata configuration error
14	Fidelity density matrix error
15	QST program error
16	Empty quantum program
17	Qubit number error, mismatch with target logic gate
18	Quantum chip Qubit topology error
19	Quantum chip calculation task program error
20	Duplicate Measure operation
21	Database operation error

Error Code	Description
22	System task status info full
23	Backend module failed to send task to backend (Communication failed)
24	Cluster computing error
25	Scheduling error, chip topology doesn't support circuit execution
26	Task configuration error
27	Unknown application ID
28	Unknown task ID
29	Parse subtask result error
30 [cite: 80]	System call timeout
31	Task has been terminated
32	URL address error
33	Parameter error
34	Quantum circuit length error
35	Chip is offline, unable to set real computing mode
36	Unknown error
37	Login password error
38	Username already registered
39	Token verification failed
40	Task query failed
41	User query failed
42	Insufficient permissions
43	API Key error
>1400	Measurement and control error (Marks hardware error)

8 Operations and Management

8.1 System Maintenance

Service Start/Stop Commands

Method 1: Users can start and stop the PilotOS system via the management page. For detailed operation methods, refer to the related instructions in "[4.4.4 Upgrade PilotOS Functional Module](#)".

Method 2: Pull up the image again. For detailed operation methods, refer to "[2.5 Service Startup and Verification](#)".

Database Backup and Recovery

The MySQL and MongoDB databases used by the PilotOS service are configured by the user. Users can choose an appropriate database backup scheme according to their own needs.

User Permission Management

For detailed operation methods, refer to the related instructions in "[4.2.3 User Management](#)".

System Upgrade Process

For detailed operation methods, refer to the related instructions in "[4.4.4 Upgrade PilotOS Functional Module](#)".

8.2 Troubleshooting

List of common problems and their solutions.

Problem List:

- **Tasks stuck in "Sent" status**

Possible Reason: Disconnected from the measurement and control system.

Solution: Check the network connection and restart the measurement and control service.

- **Database connection failed**

Possible Reason: Redis is not running.

Solution: Execute `systemctl start redis`.

- **Compilation failed**

Possible Reason: Syntax error.

Solution: Check the format of the submitted task circuit.

- **Slow page loading**

Possible Reason: Cache buildup.

Solution: Clear browser cache or server-side cache.

8.3 Deployment FAQs

This section summarizes common problems and corresponding solutions during the system deployment process to facilitate quick troubleshooting and locating.

8.3.1 PilotOS Service Anomaly & PilotOS License Verification Service Temporarily Inaccessible

Problem Phenomenon

After completing the deployment of PilotOS, access the system via `http://<Server IP>:10020`. When clicking “Activate License”, the following error message may appear, as shown in Figure 23:

The license verification service of Origin PilotOS is currently unavailable.
The system is unable to verify the license at this time.
Please try again later, or restart the related service and contact your system administrator if the issue persists.

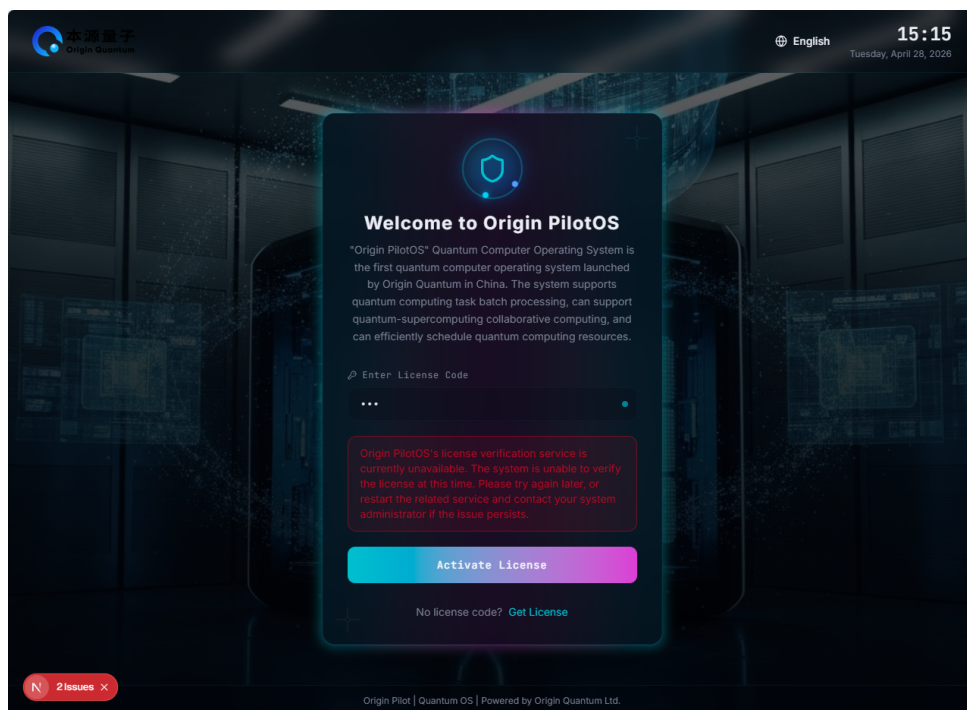


Figure 23: PilotOS Service Anomaly

Possible Problem Causes

- IP configuration error
- MySQL table structure not initialized or initialization error
- MongoDB connection error

Troubleshooting Steps

According to the config.json configuration, sequentially check the connectivity of configured MySQL and MongoDB. The following commands need to be executed on the host machine:

1. IP Configuration Check

Check the database IP configured in config.json. Do not configure the IP as 127.0.0.1, because the image operates in an isolated environment. Configuring 127.0.0.1 sets the database within the image rather than the host machine; it must be configured as the host machine's IP.

2. Check MySQL Connectivity

This step is divided into server-side checking and client-side checking to ensure the MySQL service is running normally and accessible remotely.

(1) Server-Side Check

```
1 # 1. Check MySQL service status
2 sudo systemctl status mysql
3 # If not started, start the service
4 sudo systemctl start mysql
5
6 # 2. Check MySQL listening port and address
7 ss -lntp | [cite: 82] grep 3306
8 # Note: If it only binds to 127.0.0.1, remote access is not
   allowed
9
10 # 3. Check configuration file
11 sudo nano /etc/mysql/mysql.conf.d/mysqld.cnf
12 # Verify the following:
13 # bind-address = 0.0.0.0           # Supports remote access
14 # skip-networking                 # Should be commented out or
   ensured not enabled
15
16 # 4. Check if firewall allows 3306
17 sudo ufw status
18 sudo ufw allow 3306/tcp
19
20 # 5. Log in to MySQL to check user permissions
21 mysql -u root -p
22 # View users and accessible hosts
23 SELECT User, Host FROM mysql.user; [cite: 83] # Ensure the
```

```

    Host for the user needing remote access is '%' or the
    client IP
24 # View database permissions
25 SHOW GRANTS FOR '<Username>'@'<Client IP or %>'; [cite: 84]

```

(2) Client-Side Check

```

1 # Confirm smooth network between client and server
2 ping <MySQL Server IP>
3
4 # Attempt to connect to MySQL
5 mysql -h <MySQL Server IP> -P 3306 -u <Username> -p
6
7 # View if the configured database exists after initialization
8 SHOW DATABASES;
9
10 # Enter the target database
11 USE <Your Database Name>; [cite: 85] # View table status to
    ensure successful initialization
12 SHOW TABLE STATUS;
13 # Terminal should display something like "26 rows in set",
    otherwise initialization failed

```

If the client doesn't display 26 tables above, you need to re-execute step [2.4.1](#).

3. MongoDB Database Deployment and Permission Configuration (Execute the following on the MongoDB server)

(1) Check MongoDB Service Status

```

1 sudo systemctl status mongod
2 sudo systemctl start mongod

```

(2) Check or Create Pilot Database

```

1 mongosh
2 show dbs
3 use Pilot
4 db.init.insertOne({init: 1})

```

(3) Create User to Access Pilot Database

```

1 use Pilot
2 db.createUser({
3   user: "username",
4   pwd: "password",
5   roles: [

```

```
6     { role: "readWrite", db: "Pilot" }
7   ]
8 })
```

(4) Enable Authentication Mechanism

Edit configuration file:

```
1 sudo nano /etc/mongod.conf
2     [cite: 86]
```

Add content:

```
1 security:
2     authorization: enabled
```

Restart service:

```
1 sudo systemctl restart mongod
```

(5) Configure Remote Access Permissions

Modify configuration file:

```
1 net:
2     bindIp: 0.0.0.0
```

Open firewall port (ignore if firewall is disabled):

```
1 sudo ufw allow 27017/tcp
```

(6) Remote Connection Test

```
1 mongosh "mongodb://pilotUser:StrongPassword123!@ServerIP
:27017/Pilot?authSource=Pilot"
```

Verification Method

After confirming MySQL and MongoDB connectivity, you need to restart the PilotOS image: Execute the following commands in the pilotos-4.0 folder to restart PilotOS image services:

```
1 sudo docker compose down
2 sudo python3 AutoDeploy.py start
```

After restarting, access <http://<Server IP>:10020> again and verify the license. If the above problem is still not resolved, please contact operations personnel through the official website.

8.3.2 License Verification Server Connection Error, Please Check Network

Problem Phenomenon

Clicking activate license prompts **Failed to connect to the license verification server. Please check your network settings.**, as shown in Figure24.

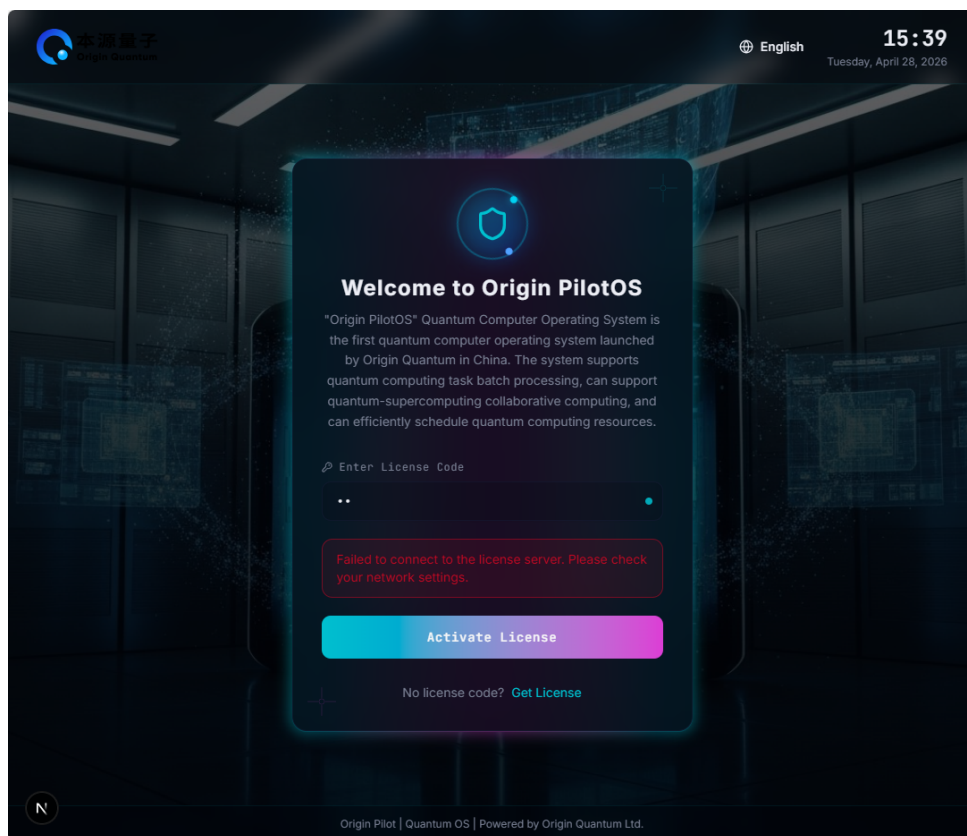


Figure 24: License Server Connection Anomaly

Problem Cause

Accessing the PilotOS license verification server requires the local machine to be able to connect to the public internet. If this problem occurs, it's a local network issue, please check if your local network is normal.

Verification Method

After confirming the local machine can connect to the network, execute the following commands inside the image to verify the local machine can access the PilotOS license verification server. First, enter the PilotOS image, then use the curl tool to test connectivity.

```
1 sudo docker exec -it pilotos-server bash
2 curl -v -k -X POST https://qcloud.originqc.com.cn/api/license/authLicense.json -
  ↪ H "Content-Type: application/json" -d '{"test":1}'
```

If the connection is normal, the server will return the following information:

```
1 {"success":false,"code":60001,"message":"Invalid License, please input correct  
↪ License","obj":null}
```